Raspberry Pi based Cyber Physical System for Driver Monitoring

BY

CHEE HAO JIE

A REPORT

SUBMITTED TO

Universiti Tunku Abdul Rahman

in partial fulfillment of the requirements

for the degree of

BACHELOR OF INFORMATION TECHNOLOGY (HONOURS) COMPUTER ENGINEERING

Faculty of Information and Communication Technology

(Kampar Campus)

June 2025

COPYRIGHT STATEMENT

This Final Year Project proposal is submitted in partial fulfilment of the requirements for the degree of Bachelor of Information Technology (Honours) Computer Engineering at Universiti Tunku Abdul Rahman (UTAR). This Final Year Project proposal represents the work of the author, except where due acknowledgment has been made in the text. No part of this Final Year Project proposal may be reproduced, stored, or transmitted in any form or by any means, whether electronic, mechanical, photocopying, recording, or otherwise, without the prior written permission of the author or UTAR, in accordance with UTAR's Intellectual Property Policy.

ACKNOWLEDGEMENTS

I would like to express my sincere thanks and appreciation to my supervisor, Dr Teoh Shen Khang for providing me this bright opportunity to engage in this IoT project. This project shall be my first step to establish a career in embedded systems and IoT technology field. I truly appreciate all the guidance, support, patient and valuable ideas that Dr Teoh has shared throughout this project. A million thanks to you.

A very special thanks to my parents and family for supporting and motivating me throughout my academic journey. Their patience, sacrifices and love is the driving force for me to keep moving and achieve success in life.

ABSTRACT

The revolution of Internet of Things (IoT) has transformed the way humans interacts with technology. This concept involves connecting devices via the internet, which allow for more efficient real-time communication and interaction regardless of distance. In the automotive field, the integration of IoT devices plays an important role in improving driver experience and safety. Nowadays, driver drowsiness and drunkenness are the two main causes of accidents worldwide. However, many existing solutions address these problems separately and leave gaps in protection. In address of this problem, a more comprehensive IoT-based driver monitoring system by using Raspberry Pi will be presented through this report. The proposed system combines several safety features into a single platform. For driver authentication, face embedding techniques is applied to verify the vehicle owner. Drowsiness detection is analysed through four criteria including eye landmarks, mouth aspect ratio, nose length ratio, and supported by a pretrained model to improve accuracy. Lastly, alcohol impairment is monitored through a gas sensor that measures the alcohol concentration in the air. The proposed system is also integrated with a buzzer, where it is activated upon the detection of any signs of fatigue or drunkenness to alert the driver immediately. To make the system more practical, a mobile application is developed to provide a simple and user-friendly interface. This application serve as a medium to display the output of the system where users are able to monitor the driver status, authentication results, and alerts in real time. This system aims to showcase how IoT hardware and mobile technology can be integrated to address multiple road safety issues at once and enhance driver safety.

Area of Study: Internet of Things, Cyber Physical System

Keywords: Driver Monitoring System, Drowsiness Detection, Drunkenness Detection, Face Recognition, Mobile Application

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF SYMBOLS

℃	Celsius
Ω	Ohm (resistance)
%	Percentage

LIST OF ABBREVIATIONS

IoT	Internet of Things
GPS	Global Positioning System
ECU	Electronic Control Unit
EAR	Eye Aspect Ratio
MAR	Mouth Aspect Ratio
NLR	Nose Aspect Ratio
ED	Euclidean distance
FSS	Force Sensitive Sensor
CNN	Convolutional Neural Networks
TFLite	TensorFlow Lite
FCNN	Fuzzy convolution neural network
ECG	Electrocardiogram
EEG	Electroencephalography
OS	Operating System

Chapter 1

Introduction

In this chapter, we present the background study of this project, problem statement and motivation of our research, objectives, project scope and contributions to the field, and lastly outline the report organization.

1.1 Problem Statement and Motivation

In recent years, the number of accidents in Malaysia has growth rapidly. Driving has become a necessity for many due to its conveniences and efficiency. However, lacking self-control and awareness may exposed oneself in danger while driving. In this context, the common causes of these accidents are mostly due to human behaviour such as drowsiness and drunk driving. This is supported by [1], where the Ministry of Transport claimed that Malaysia is undergoing an average of one accident per minute in 2022 and a study by the Malaysian Institute of Riad Safety Research (MIROS) found that most of these accidents were caused by human behaviour. Another study [2] further highlighted the severity of car accident, where Malaysia is listed as the third highest in terms of road fatalities rate with 22.48 road deaths per 100,000 people. These statistics are awful, as many lives are lost each year as the driver fail to restrict their own behaviour while driving.

Table 1.1.1 Road fatalities per 100,000 people in different countries

Country	Road deaths per 100,000 people	Country	Road deaths per 100,000 people
1 Saudi Arabia	35.94	11 United States	12.67
2 Thailand	32.21	12 Georgia	12.41
3 Malaysia	22.48	13 Oman	10.59
4 Kuwait	15.43	14 Romania	10.29
5 Colombia	15.42	15 New Zealand	9.57
6 Chile	14.91	16 Poland	9.38
7 Argentina	14.06	17 United Arab Emirates	8.91
7 Panama	13.92	18 South Korea	8.59
9 Mexico	12.78	19 Greece	8.31
10 Kazakhstan	12.68	20 Portugal	8.20

1.1.1 Problem Statement

Talking about the root cause of car accident mentioned earlier, despite road condition or weather condition, most of the time self-behaviour is always the hardest to control. As we know, drowsiness refer to the feeling of tiredness or excessively fatigued in daytime [3], this happens when someone is not getting enough rest. While drunk driving is when someone driving while consume alcohol that exceed the safety limit. The fundamental problem of these issues is that this behaviour not only harm the driver and passengers but also other road users. In case a driver is undergoing fatigue status while driving alone, without proper self-monitoring, it is extremely dangerous.

Furthermore, fatigue is a physiological phenomenon which is unpredictable and uncontrollable. This have indirectly left a potential risk for safe driving. Same scenario when comes to drunk driving. Upon drinking alcohol, it may lead the driver to various symptoms including dizziness, decreased alertness, aggressive driving, slow reaction, fatigue, etc. Most of the time, a drunk person will not claim himself to be drunk, as alcohol affect the ability of individuals to make proper decision-making and self-control. In this context, if a drunk individual drives without acknowledgement that he is drunk, definitely the risk of road accident is more likely to happen.

Other than human behaviour, the issue of unauthorized use of vehicle should also raise concern. Without proper driver verification, there is no guarantee the system is monitoring the right person behind the wheel. Thus, a driver monitoring system that equip with authenticating feature shall provide a more solid and reliable driving monitoring solution.

After reviewing several safety risks, let's analyse the existing detection system. Despite there are various research effort aim to address the drowsiness issue, but most of it focus only on one risk factor at a time. This will leave gaps in driver safety, as the system designed for drowsiness cannot detect alcohol impairment or authenticate the driver at the same time. Other than that, some existing drowsiness detection system also proposed ineffective approach to alert or notify driver. For example, a simple blinking of LED upon drowsiness detection. In addition, certain system rely on complex physiological measurement such as EEG or ECG sensor. While this method can achieve high accuracy in controlled experiments, but they are often impractical and uncomfortable for daily use, making them unsuitable for real-world scenario. Therefore, there is a need for more comprehensive driver monitoring system that are capable of

performing multiple functionalities at once while providing effective alerts to reduce the risk of accidents.

1.1.2 Motivation

The motivation of this project is to design an integrated driver monitoring system that can address multiple safety concerns in a single platform. The system combines several sensors to detect driver drowsiness and alcohol impairment and also equipped with authentication feature to recognise the vehicle owner. The fundamental aim of this project is to enhance road safety by continuously monitoring the driver's condition and providing alerts to notify the driver. At the same time, the authentication feature provide an additional layer of protection to prevent unauthorized access of vehicle. By integrating these functions into a single prototype, this project shall provide a comprehensive solution that protects both drivers and vehicle owners effectively.

1.2 Objectives

The aim of this project is to develop a driver monitoring system that authenticate driver identity and monitor driver status. Below are several objectives that will be achieved through this project.

- **Develop a driver authentication system.**
  A driver authentication system is developed to ensure the system able to recognize the authorized driver before the monitoring begins. The system will capture the driver's face and check if it matches the registered driver. The verification step will repeat multiple times, and the driver is consider authorized only if he passes all verification attempts.

- **Implement a real time monitoring system to detect driver drowsiness.**
  The driver drowsiness detection system able to monitor driver alertness in real time. The algorithm utilizes the camera to track the signs of fatigues from different facial features such as eye blinking, mouth movement and head position.

- **Implement alcohol impairment system**

The alcohol detection able to detect whether a driver is under the influence of alcohol. It utilizes a gas sensor to measure the concentration of alcohol from the driver's breath.

1.3 Project Scope and Direction

The scope of the project includes the development of a driver monitoring system using Raspberry Pi as the main processing unit. The system focuses on three functions including driver authentication, drowsiness detection and alcohol detection. It is designed to works together with a self-developed mobile application that serves as both a control unit and medium to display the monitoring result. The prototype is designed to tested in a controlled indoor environment that simulates the condition of a car cabin to ensure the system functions optimally. It is mainly targeting private vehicle owners who seek for practical tool to improve driving experience and safety. However, the system performance is downgraded by certain factors including lighting conditions, air flow and camera positioning. Overall, the project scope aims to demonstrating the functionality of addressing unauthorized usage, driver drowsiness, and alcohol impairment problem, rather than delivering full-scale commercial automotive solution.

1.4 Contributions

People are always looking for practical IoT solutions that not only solve their current problems but can also adapt to future needs. In the area of driver monitoring, although there exist many solutions but some of them are too complex, ineffective in alerting driver and only focus on solving single issue. Also, most of them are difficult to customize and present via a user-friendly interface, which make them less effective for real-world use.

With this project, several contributions are made to address those challenges. First, the system brings together the important functions like driver authentication, drowsiness detection and alcohol detection into a single prototype. This integration demonstrate how multiple safety features can be integrated together to make the monitoring system more robust. Other than that, instead of using over complex method for detection, it uses camera-based and sensor-based techniques that are more cost effective and practical for everyday use. Third, it improve alert effectiveness by combining buzzer with mobile application ringing. This approach ensure the driver is alert immediately at critical moment. Finally, the project highlight how IoT and mobile

technology can work interchangeably which provide a foundation for future development of automotive safety system.

1.5 Report Organization

This report is structured into several chapters to outline the development process of a driver monitoring system.

- Chapter 1: Introduction
  In this chapter, an overview of the project was performed. This include background study, problem statement, motivation, project scope and research objective of the project. This chapter shall provide the reader with a kickstart on how the project will be carried out.

- Chapter 2: Literature Review
  This chapter discusses the previous research and methods proposed by other authors on driver drowsiness detection. Several research papers are review thoroughly to understand the methods the used. Also, the strengths and weaknesses of each previous study are analysed. The proposed solutions are also listed below to overcome the weakness of each study.

- Chapter 3: System Methodology/Approach
  In this chapter, the methods used to carry out this project was discussed. This chapter discussed the system architecture, use case diagram and activity diagram of the project.

- Chapter 4: System Design
  This chapter outline the system architecture including hardware architecture and software architecture. All the system components specifications and how the circuits connection are discussed in this chapter. Also, the important algorithm such as face authentication and drowsiness detection system are explained in detail.

- Chapter 5: System Implementation
  In this chapter, the hardware setup, software setup and the configuration are presented. Also, how the system is operated and the implementation issue.

- Chapter 6: System Evaluation and Discussion

  This chapter will discuss the result by applying various test case. Also discuss the project challenges and evaluate whether the objectives are met.

- Chapter 7: Conclusion and Recommendation

  This chapter summarize the whole project and provide recommendation for future improvement.

Chapter 2
Literature Review

2.1 Previous Works on IoT-Based Driver Monitoring System

2.1.1 IoT-Based Smart Alert System for Drowsy Driver Detection

In the project proposed by [4], the author have address driver fatigue as the potential issue leading to global impact of road accident due to inadequate driving. To overcome this issue, the paper focuses on constructing smart or intelligent vehicles that automatically detect and prevent driver drowsiness.

The method used by [4] involved using eye blinking concepts accompanying with Eye Aspect Ratio (EAR) and Euclidean distance (ED) of the eye to analyse the drowsy expression of a driver. The working mechanism of the system is by using the threshold value of EAR. Suppose a driver does not show any effect of fatigue, the EAR value is always above the threshold value. On the contrary, if the driver fall asleep, then the value will fall below the given threshold indicating drowsiness of the driver is detected. To accurately calculate the threshold based on the driver eye blinking, Raspberry Pi3 camera has been used to capture the eye landmark.

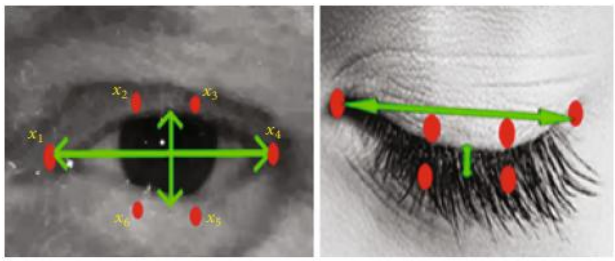


Figure 2.1.1.1 Eye landmark

When fatigue is detected, the system speaker will issue a speaking voice to alert the driver. If the driver is still not awake after the system voice, then the IoT module will send an e-mail to the authorized person, so that this person can phone ringing the driver to further alert him. Also, the system is integrated with crash sensor and Force Sensitive Sensor (FSS). In case the sensor detect a collision with other vehicle, it will send the collected data to the car owner, at the same time the location of where the collision happened is also sent to the nearby hospital.

2.1.2 Neuromorphic Sensing for Yawn Detection in Driver Drowsiness

In Kielty et al. [5], the authors proposed a vision-based approach to classify drivers' drowsiness based on yawning detection. The system uses a camera to observe the driver's facial features and image processing software to identify the mouth region. In this case, the detection on detecting excessive mouth opening as the key indicator of yawning.

For obtaining robustness, the dataset was pre-processed to choose appropriate frames with a clear view of yawning and compared with neutral face expressions. Then CNN structure was utilized for learning region differentiated features so that automatic identification of yawns could be done without the need for handcrafted features. This approach allows the system to deal with variations in the appearance of the driver, e.g., changing facial features or illumination conditions, more effectively than previous geometry-based approaches. The performance of the proposed method was evaluated with test subsets of the datasets and gave approximately 98% accuracy along with great precision and recall values. These results indicate that the system performs effectively under controlled conditions in recognizing yawning.

2.1.3 Driver Drowsiness Detection Model Using Convolutional Neural Networks Techniques for Android Application

Research did by [6] address the problem of microsleep and drowsiness issue facing by the driver that may lead to road fatalities. The proposed system aims to integrate a system that is able to detect microsleep and drowsiness by using the Convolutional Neural Networks (CNN) method. CNN is known as a machine learning model, which use deep learning algorithm to train and capture specific patterns within images from a large dataset. As mentioned by [6], Saifuddin Saif, et al used a dataset called National Tsing Hua University (NTHU), in which the dataset contains people from different ethnics who shows the behaviour of drowsy such as yawning, slow rate blinking, drowsy eyes, etc. under different lighting condition (Figure 2.3).

Figure 2.1.3.1 Sample images of NTHU dataset

After that, the dataset is fed into the model to get trained. In the system, a camera is used to capture the driver facial expression and fed it into the detecting algorithm to identify the driver drowsiness status. An android application is also developed as a medium to feedback. If the data match to any one of the drowsy behaviours, then the android application will send notification through both visual and audio messages to alert the driver.

2.1.4 Real-Time Drowsiness Alert System from EEG Signal Based on FPGA

According to [7], the issue of road accidents has arisen which is mainly due to driver drowsiness. The consequences is it has led to numerous injuries and fatalities. Although there exist vary of drowsiness detection methods such as behavioural-based, vehicle-based and physiological parameter-based, but these methods are hard to implement in real-time approach. In this context, [7] has proposed a drowsiness alert system by using Electroencephalography (EEG) signal. Based on [7], so far this approach is the fastest among other method as EEG involve capturing the brain electrical activity. It is developed using FPGA platform that support real time and high-speed processing in nanoseconds. For the implementation, EEG sensor is place around the driver's scalp to measure brain activity signal. Then this signal is sent to control unit to calculate the respective frequency. The way to detect the drowsy state by using the EEG signal is based on the frequency band in Theta region (Table 2.1.4.1).

Table 2.1.4.1 Frequency Band and Mental State of EEG Signal

1	**Beta region**	14-30 Hz	Awake with mental activity
2	**Alpha region**	8–13 Hz	Awake and Resting
3	**Theta region**	4–7 Hz	Drowsy state
4	**Delta region**	0.5–4 Hz	Dreamless sleep

After obtaining the respective frequency, the duration is taken into account with threshold value of 10ns. Suppose the system capture two repeated theta and the difference between these theta frequencies is 10ns, then drowsiness is detected. A buzzer will then issue an alarm to alert the driver.

2.1.5 Real-Time Drowsiness Detection System for Driver Monitoring

In project developed by [8], the author has outlined the importance of drowsiness detection system for a vehicle to ensure the safety of driver as well as other vehicle. Besides, [8] also argue on the current problem of the most drowsiness detection system such as inefficient drowsiness detection system, lack of real time feature, and lack of notification support.

In the proposed system, they use the combination of three approaches including physiological approach and vehicle-based approach to develop the system. In physiological approach, eye blink sensor is embedded in a wearable spectacle and heartbeat sensor is wear on the driver's finger. Both of the sensor are wearable and attach directly to the driver's body. Figure 2.4 show the overall system architecture.

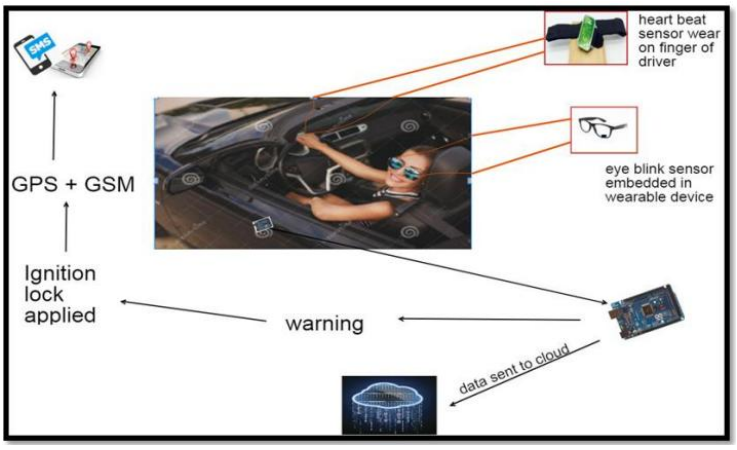


Figure 2.1.5.1 Sample system Architecture

Eye blink sensor and heartbeat sensor are working together, in which three different ranges of proximity and hear beat rate are used to identify the driver fatigue status. First condition, if the proximity is between 3.00>=proximity<=4.00 and heartbeat rate is between 70>=r<=84, it indicate the driver is currently awake, the alarm will sound for 4 seconds. Second stage, if the proximity is between 2.00>=proximity<=2.90 and heartbeat rate between 65>=r<=69, it show that the driver is drowsy, and the alarm is sound for 8 seconds. Third stage, if the proximity is between 1.00>=proximity<=1.90, it means the driver is sleepy, and the alarm will continuously sound.

Furthermore, to enhance the safety of driver during third stage, an ignition lock control system is applied. This system is a vehicle-based approach, where it works by slowing down the speed of vehicle gradually every 4 seconds. This is an importance approach to give proper reaction time to other vehicle instead of stopping the car directly. After the car is completely stopped, then a message contain the location detail which fetch from the GPS module is sent to the driver friends or relative one, so that they can help phone ringing the driver to wake him up.

2.1.6 Non-Interference Driving Fatigue Detection System Based on Intelligent Steering Wheel

[9] propose a non-interface fatigue detection system which an electrocardiogram (ECG) acquisition device is embedded within the steering wheel, along with ECG-based fatigue detection model. The working mechanism is, when the driver having their hand holding on the steering, the system will collect the ECG signal also known as palm signal, and this signal is sent to the fatigue detection model to analyse the driver fatigue status.

The fatigue detection model include a simulation generation module based on a Cycle-Generative Adversarial Network (CycleGAN) and fuzzy convolution neural network (FCNN). The ECG signal data is fed into simulation generation module to convert the ECG signal to a chest signal which is clearer. After that the generated chest signal are transmitted to FCNN to search for the matching fatigue data.

2.2 Strengths and Limitation of Previous Studies

2.2.1 IoT-Based Smart Alert System for Drowsy Driver Detection

For the driver monitoring system developed by [4], one of the strengths is the system provide a real time monitoring feature. This can be observed through the use of a camera to monitor the driver's eye movement. This approach is straight forward and easier to be implemented in compared to others, as the fastest way to identify drowsiness is through eye blinking. In addition, the use of camera can enhance driver experience, as there is no physical contact and doesn't require any interaction. Also, the crash sensor and FSS sensor integrate on the system has further enhance the overall performance of the monitoring system. Those sensors reflect on the collision event, and it issue notification such as the accident location. In this context, the monitoring system not only focus on preventing accidents, but it also responds to accidents which ensure driver safety. The system also provide effective driver alert, where a buzzer is used to alert the driver when drowsiness is detected.

For the limitation of [4] project, environmental condition is crucial one to ensure the system functioning well. As the system relies on the camera to capture the visual data, lighting conditions such as night driving can interfere the camera to capture the eye movement effectively. Other than environmental conditions, the proposed method is less effective in classify drowsiness. This is because it only focused on eye blinking detection and unable to detect other possible fatigue signs like yawning.

2.2.2 Neuromorphic Sensing for Yawn Detection in Driver Drowsiness

The Kielty et al. [5] suggested approach is better in various aspects and appealing for drowsiness detection. Since yawning is one of the common signs of fatigue, the system provides a simple but effective way to monitor the alertness of drivers. It is also unobtrusive since it utilizes a camera without applying other body sensors, making it more manageable for drivers. Applying a CNN method does not depend on manually crafted expressions such as MAR and is able to learn informative features automatically, thus being more able to track changes in facial structure and illumination. Under controlled conditions, the CNN model has achieved high accuracy nearly 90% accuracy which show that this method is reliable.

But the technique has its serious shortcomings. It is based solely on yawning as a measure of drowsiness, which may miss a case where a driver is fatigued but does not yawn. Other actions such as chatting or laughter may also be mistaken for yawns and trigger false alarms. At the same time, the trained model is only focus on yawning detection instead of other fatigue signed like closed eye or head bend down. Although the system achieves high accuracy on test sets, but its performance in real driving situations might degraded due to adverse lighting, camera position, or partial occlusion of the mouth. The study also focused predominantly on detection and did not describe how drivers would be alerted after drowsiness was detected, limiting its direct use as an overall safety system.

2.2.3 Driver Drowsiness Detection Model Using Convolutional Neural Networks Techniques for Android Application

Based on the project develop by [6], one of the strengths of the project developed is having high accuracy with lightweight model. The proposed CNN model support lightweight with maximum size of 75 KB, while providing sufficient performance to detect drowsiness and offer smaller storage space. Based on the result, the system achieved more than 83% accuracy when comparing various categories of data that passed to CNN-based train model. Another strength of the project is real time feature. Real time is indeed an important measurement to prevent missing out any critical moment. The system support real time data capturing by continuously monitor on the driver expression, which is crucial to ensure driver safety. Besides, this project also design to have low computational requirements in terms of processing power and memory requirements. This feature allow the system to be efficiently integrated into a smaller embedded system. Last but not least, the system is integrated with an android application which will issue alarm that alert the driver. This measurement can effectively prevent driver from become fatigue. Last but not least,

However, there also exist some limitation to the proposed system. One of the potential limitations is sensitive to lighting condition. In this scenario, the system accuracy are more likely to drop when the environment is in low light, especially during night or when wearing sunglasses. Based on the result, when there is uneven lighting or darkness, it may cause lack clarity to the image captured, and leading error rate to increase by 3%. Another limitation is the facial features dependencies. This situation happen when driver face is partially obscured,

mainly due to sunglasses or facemask. In this context, the system is having difficulties to identify driver expression and leading to inaccurateness of result.

2.2.4 Real-Time Drowsiness Alert System from EEG Signal Based on FPGA

In the proposed system introduced by [7], one of the strengths is the system offers high precision in detecting drowsiness. This is because the approach used by [7] is EEG signal which is obtained through driver's brain activity. Since drowsiness actions are reflect directly through brain activity, by monitoring on EEG it can predict drowsiness more effectively instead of physical action such as eye closure, yawning, head signs and so on. Another strength of the system is real-time processing. As the system is driven by FPGA, this allow fast processing of the EEG data. Besides, the parallelism characteristic of FPGA allow the system to process complex instruction such as EEG signal. Also, high processing speed of FPGA minimize the time needed to analyse the drowsiness and feedback. This is essential to detect drowsiness and alert the driver immediately to prevent accidents.

For the limitation of the proposed system, the driver need to place ECG sensor on their scalp area, as the ECG signal receives through driver' brain. In this case, it will definitely be uncomfortable for most drivers, especially in long drive situation. Furthermore, complex setup is another potential problem. The sensor has to be placed precisely at a specific part in order to receive an accurate signal. The deployment of the system seems to be easy for those with specific knowledge but unfriendlier for those who don't. In this scenario, ensuing the sensor to function is challenging for a driver.

2.2.5 Real-Time Drowsiness Detection System for Driver Monitoring

The proposed system introduced by [8] has introduced several strengths. First, is the use of sensors such as eye blink sensors and heartbeat sensors. Both of these sensors are directly embedded on wearable glasses and driver's fingers which significantly enhance the accuracy of the retrieved data to analyse the driver drowsiness. Besides, the capability of gradually reducing the vehicle speed is also a good design to prevent the vehicle from continuing to accelerate in dangerous conditions in case the driver cannot wake up on time. Another strength is the real time detection feature. The system is able to provide immediate feedback and provide necessary measurement such as alarm, sending notification, slowing down the car and ignition

clock. In this scenario, real time plays an important role in ensuring the safety of the driver, without missing out any particular moment.

For the weakness of the proposed system by [8], it too relies on the driver to use the wearables. Although physiological detection system does provide accurate data, we should also consider the driver experience and suitability when wearing those sensors. For example, imagine that every time before driving the car, the driver has to put those sensors on, which is very tedious and uncomfortable. Furthermore, physiological detection might not function as expected especially in the situation such as the driver hand or car vibration that will interfere the heartbeat sensor leading to false decision made by the system.

2.2.6 Non-Interference Driving Fatigue Detection System Based on Intelligent Steering Wheel

For the project developed by [9], one of the strengths is it support real-time detection. The system will continuously capture the ECG signal through the ECG acquisition device that is embedded within the steering wheel. With real time feature, the data updated is always the latest one which is more accurate. This feature can effectively capture the signal without missing out any signal that might reflect driver fatigue. Real-time detection playing a crucial role to provide immediate alert when drowsiness is detected. Furthermore, this system offer certain degree of conveniences as the device is directly implemented inside the steering wheel. In this context, the driver does not need to make any direct interaction such as wearing it or set up which enhance the driver experience.

In terms of limitation, the crucial one is the dependencies on steering data. As the proposed system only use ECG signal to identify driver fatigue, in case the steering data is inaccurate, then it might cause the system to make the decision wrongly. For example, road irregularities, such as pothole or hump may influenced the quality of the data. This interference will interfere the accuracy of data, which increase the difficulty for the system to identify between normal driving or fatigue condition. Other than that, the system is not ready with drowsiness detection step. Missing this feature will genuinely put driver life at risk.

2.3 Summary of Strength and Limitation of Previous Study

Method	Strength	Weakness
IoT-Based Smart Alert System for Drowsy Driver Detection [4]	- Real-time monitoring - Simple and effective - High Accuracy - Camera-based monitoring - Provide driver alerts	- Focus only on eye blinking - Environment condition such as lighting condition
Neuromorphic Sensing for Yawn Detection in Driver Drowsiness [5]	- Real-time monitoring - High accuracy - Camera-based monitoring - Provide notification alert	- Focus only on yawning - Risk of misclassify from laughing/talking - No driver alert described - Environment condition such as lighting condition
Driver Drowsiness Detection Model Using Convolutional Neural Networks Techniques for Android Application [6]	- Real-time monitoring - Lightweight model - High accuracy - Low computational requirement - Driver alert - Integrated with Android App	- Environment condition - Facial features dependencies such as sunglasses
Real-Time Drowsiness Alert System from EEG Signal Based on FPGA [7]	- Fast processing speed via FPGA - Support real-time - Driver alert - Direct measures brain activity, high accuracy in detecting fatigue	- Relies on wearables - Affect drive experience - Complex setup and positioning
Real-Time Drowsiness Detection System for Driver Monitoring [8]	- High Accuracy - Driver alert - Real-time feedback with alarm, notification and vehicle slowdown	- Relies of wearables - Affect driver experience - Environmental conditions such as road irregularities

Non-Interference Driving Fatigue Detection System Based on Intelligent Steering Wheel [9]	- Support real time - Convenience, no driver interaction needed	-Environmental conditions such as road irregularities - Dependencies on ECG signal - No driver alerts

2.4 Proposed Solutions

First, the missing drowsiness prevention step in [9] is a crucial limitation that need to be considered. One has to acknowledge that missing this step is crucial especially when the driver is driving in drowsy state without his acknowledgement. Missing this step not only harm the driver but also other vehicles. This can be solved by using methods such as triggering the buzzer alert [7], [8] or sending notification alert [4], [6].

Besides, the proposed method by [4], [5] who only focus on single fatigue sign can be solved by integrate multiple detection features. For example, [4] who relies on facial landmark calculation can implement mouth length ratio (MLR) for yawning detection and nose length ratio (NLR) for head bend down detection. And for [5] who CNN model only focus on yawning detection, can use more complex dataset like NTHU dataset that contain more drowsy expression. With this measurement, the drowsiness detection model is more comprehensive and not solely focus on single fatigue signs detection only.

For the system proposed by [7], [8] which heavily relies on wearables can be solved by using the approach suggested by [9]. Instead of using wearables with sensors that required driver interaction, it would be better to direct embedded the sensor on the surrounding equipment. For example, instead of wearing heartbeat sensor on driver's finger [8], a more effective way is to implement directly on the steering wheel [9]. Other than that, rather than using wearables device with eye blink sensor embedded on it [8], a more convenient way is to use the camera to monitor on driver eye blinking proposed by [4] or, use CNN method to classify drowsy expression [5], [6]. By considering this adjustment, it may further enhance driver experience.

Except for that, most of the proposed methods such as [5], [7], [8], and [9] are not as user-friendly. They focus primarily on the detection process without considering how the results are being presented to the driver. But the interface is important, because it provides feedback and enables effective interaction between the system and the user. Without such features, the utility of a drowsiness detection system is lost, since deployment in real environments requires not only effective detection, but also an appropriate way of informing drivers. In order to counter such a limitation, the design of a mobile application, as indicated in [6], provides timely feedback and alarm notices, making the overall system complete and more reliable.

Last but not least, the potential limitation is the environment condition. The proposed system introduced by [8], [11] who use camera to capture the driver facial expression are more likely to face this issue. This happen when the environment lighting condition is bad especially during nighttime. To address this issue, the use of camera with infrared feature can solve this problem effectively. Infrared camera can work effectively under low light condition and potentially enhance the performance of the monitoring system.

Chapter 3

System Methodology/Approach

3.1 System Design Diagram/Equation

3.1.1 System Architecture Diagram

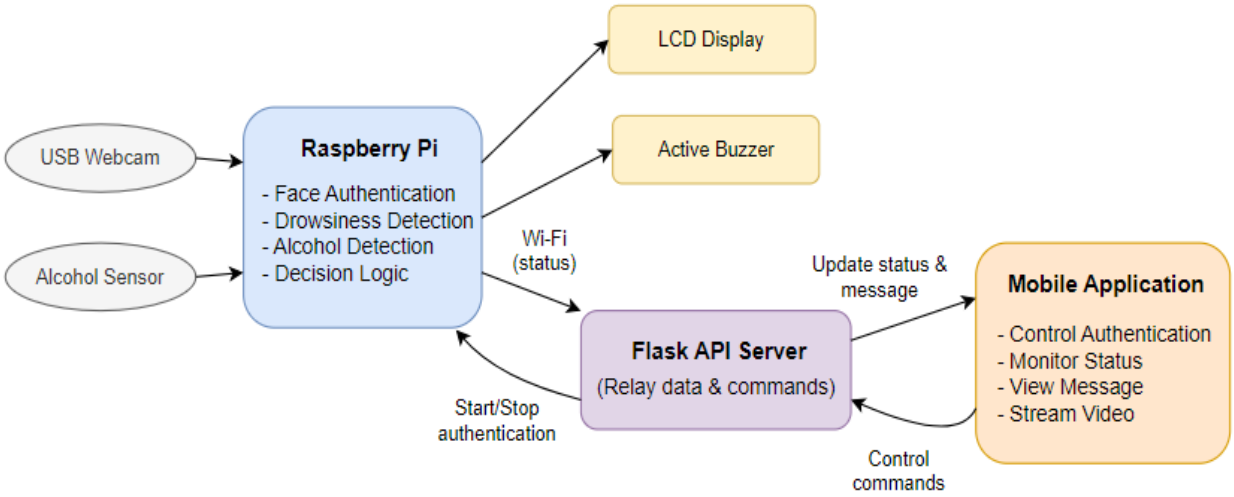


Figure 3.1.1.1 System architecture diagram

The diagram outlines the overall system architecture of the driver monitoring system with three core components. First is the Raspberry Pi which acts as the central processing unit, integrating input devices, detecting algorithm, output interfaces and also communicate with a mobile application. A USB webcam is connected to the Raspberry Pi to capture driver's face in real time which will be used for face authentication and drowsiness detection. In terms of drowsiness detection, this process relies on calculating the Eye Aspect Ratio (EAR), Mouth Aspect Ratio (MAR) and Nose Length Ratio (NLR). These values are then used to compare with thresholds over specified duration to detect eye closure, yawning or head bending down. Besides webcam, an alcohol sensor is also integrated to continuously measure the concentration of alcohol in the air around the driver. The voltage reading is then sent to Raspberry Pi to compare with the threshold.

After that, based on the detection result, Raspberry Pi executes the decision logic and trigger the alerts when necessary. In this case, two output devices are used including an active buzzer

and a LCD display. Active buzzer will issue high volume beep sound with different intervals, while LCD display will display messages such as the authentication result or warnings.

For communication with the user, the Raspberry Pi hosts a lightweight Flask API server that enable wireless data exchange across Wi-Fi. The Flask API able to relays the command from the mobile application, such as starting or stopping the authentication process. At the same time, it also able to transmit driver status and message back to the mobile app in real time. Other than that, it also support different command like video streaming which allow live monitoring of the driver through the app when necessary.

Lastly is the mobile application. It serves as the main user interface that allow driver to interact with the Raspberry Pi. The driver can control the authentication, monitor the driver status, view messages, stream video and even shut down Raspberry Pi. When the system is operating, user can start the authentication and monitoring session via the app. These commands are then sent via Flask API to Raspberry Pi and activate the camera and alcohol sensor. The app operate in real time, where the user able to view live update of the detection result. The monitoring continues until the user manually ends the session from the app.

3.1.2 Use Case Diagram and Description

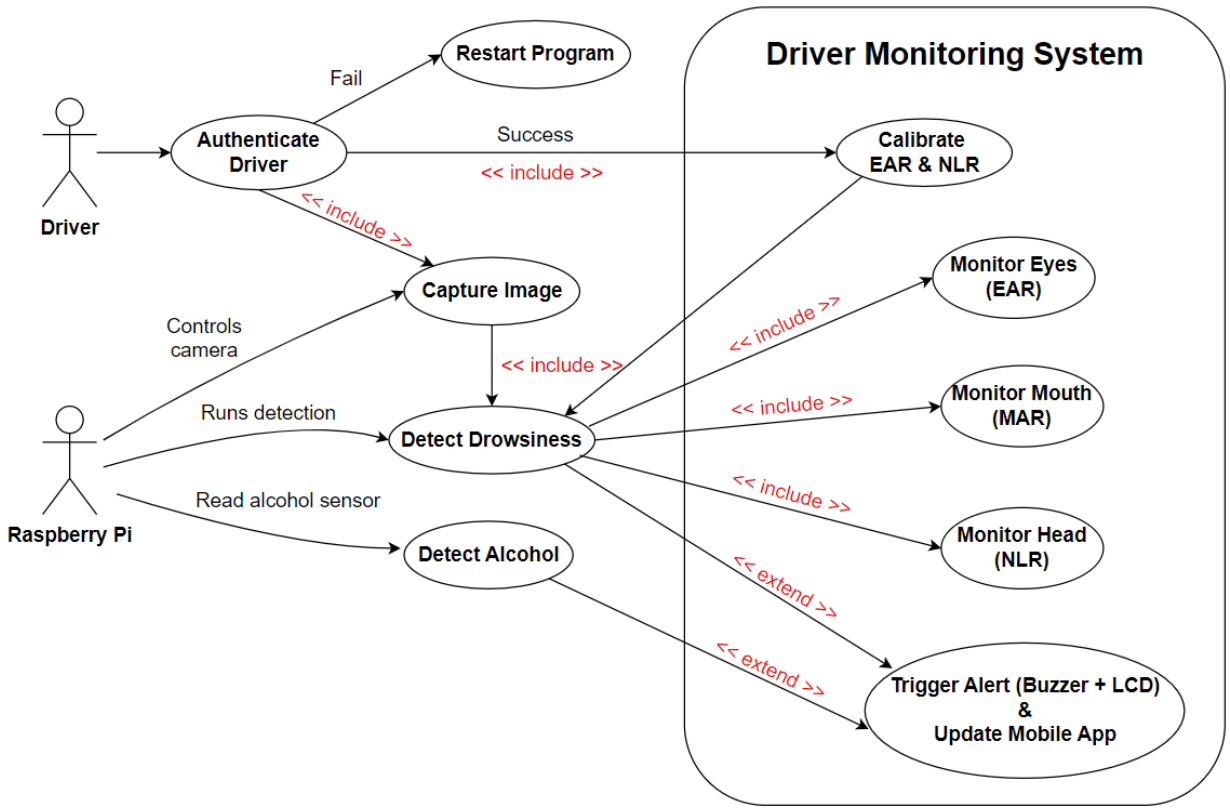


Figure 3.1.2.1 Use case diagram

The Driver Monitoring System use case diagram illustrates how the system interacts with both driver and Raspberry Pi in real world scenario. The process begins with driver performing authentication which acts as the main entry point. During this process, Raspberry Pi will start controls the camera to start capturing driver image. In this case, the Capture Image use case is used for both authentication and ongoing drowsiness monitoring. If authentication fails, driver may need to restart the program to retry the authentication. On the other hand, if successful, it proceeds to calibrate EAR and NLR threshold to personalized thresholds depends on different individuals.

After authentication and calibration processes are complete, the system will start monitoring functions. The Detect Drowsiness use case relies on three components including Monitor Eyes (EAR) to check for eye closed time, Monitor Mouth (MAR) to detect yawning and Monitor Head (NLR) to identify head bending down. If any drowsy signs are detected, it extends to Trigger Alert (Buzzer + LCD) to alert the driver and update the mobile application. At the same time, Raspberry Pi also runs Detect Alcohol use case by continuously reading the alcohol sensor. If concentration of alcohol are high, this also extends to trigger the same alert and update mechanism.

As a summary, Raspberry Pi coordinates these processes by controlling the camera, running detection, reading alcohol sensor, and sending updates. These interactions are required to enable the system to work only after authentication, then constantly monitor for drowsiness and alcohol impairment, and provide timely warning and update status.

3.1.3 Activity Diagram

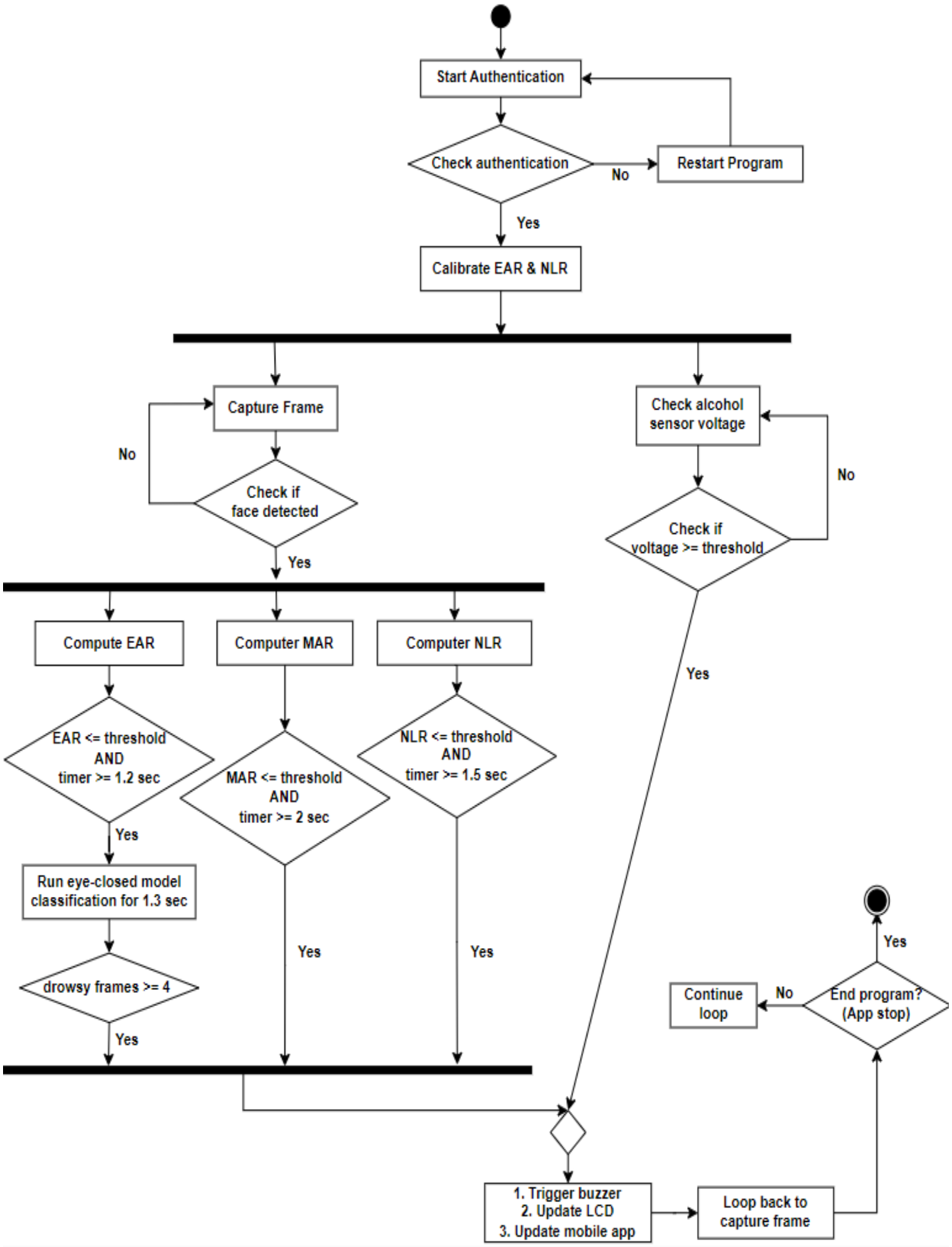


Figure 3.1.3.1 Activity diagram

The activity diagram illustrates the complete workflow of a driver monitoring system, beginning with start authentication. During authentication, the system captures multiple frames of driver's face to check if it is matched. If authentication fails, the driver may need to restart the authentication. If successful, the process moves forward to calibrate EAR and NLR to set personalized threshold for monitoring. After calibration is done, the workflow splits into two parallel processes.

The first branch starts with capturing the frame which is specifically for drowsiness monitoring. Each captured frame is checked to confirm that a face is detected. If no face is found, the system loops back to capture another frame. When a face is detected, three instances will run in parallel including compute EAR for eye closure, compute MAR for yawning and compute NLR for head bending down. Each of them is compared with respective threshold and time condition. If any condition is met, then the system will trigger an alert.

The second branch runs independently to perform alcohol detection. The system continuously reads the alcohol sensor voltage and compares it with the threshold. Voltage exceed threshold indicate alcohol is detected, and the system will trigger an alert. In both branches, triggering an alert involves activating the active buzzer, updating LCD display and sending status updates to the mobile app. The system will loop continuously for both drowsiness monitoring and alcohol detection until the program is terminated from the mobile app.

Chapter 4

System Design

4.1 System Block Diagram

4.1.1 Block Diagram for Hardware Architecture

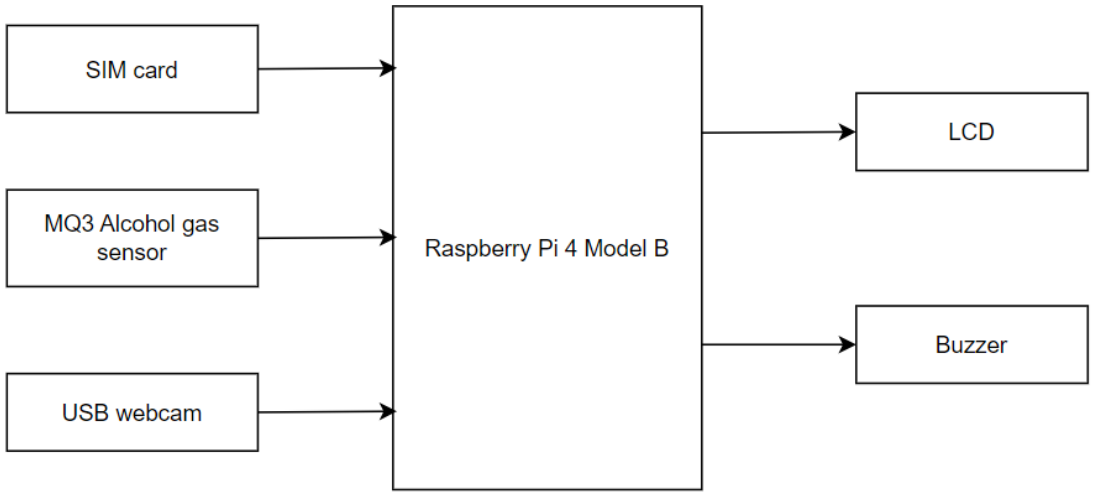


Figure 4.1.1.1 Hardware architecture

Raspberry Pi 4 Model B serves as the main processing unit for the driver monitoring system. The input devices include SIM card, MQ3 alcohol gas sensor and USB webcam. SIM cards are used for IoT connections which allow microcontroller for mobile application. MQ3 alcohol gas sensor will detect alcohol vapor in the environment and USB webcam captures driver's face for face authentication and drowsiness detection. And lastly, GPS module to receive the current location address.

On the output side, the Raspberry Pi controls both LCD display and buzzer. The LCD display serve as a feedback devices to show the driver condition while the buzzer serve as alert device to provide high volume beep sound when drowsiness and alcohol is detected. As summary, this hardware setup allows Raspberry Pi to implement visual monitoring, alcohol detection and communication, and provide real-time alerts and updates to the mobile app.

4.1.2 Block Diagram for Software Architecture

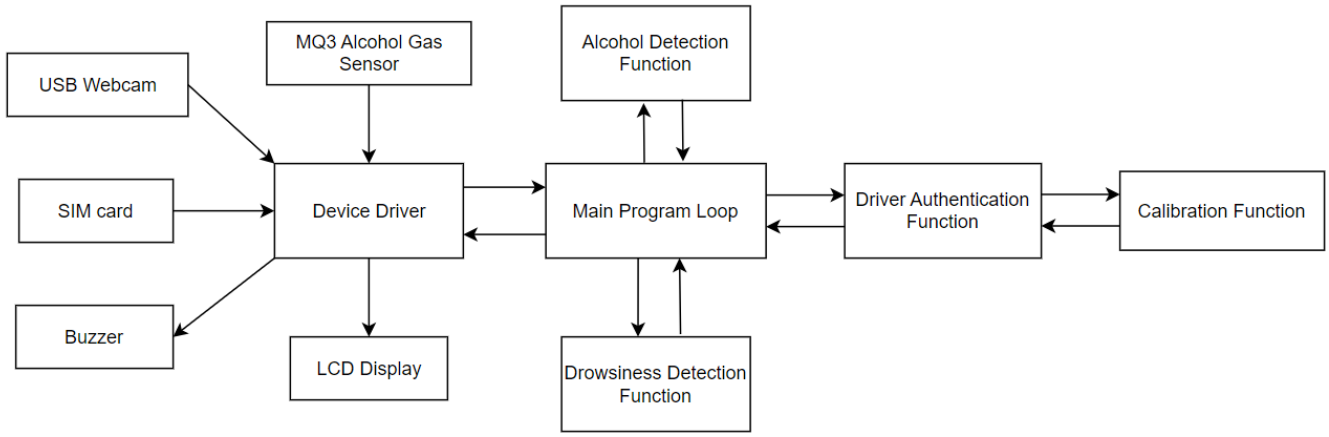


Figure 4.1.2.1 Software architecture

The software architecture is structured into two primary layers including main program loop and device driver layer. The main program loop contains all the system application logic. It first begins with driver authentication function, which utilize USB webcam to capture driver face and verify their identity. Once the authentication is successful the calibration function will take part to set personalized threshold for EAR and NLR values. After this stage the system proceed to drowsiness detection function and alcohol detection function. The drowsiness function classify drowsy signs in terms of eye closure, yawning and head position using the webcam, while the alcohol detection function monitor alcohol concentration from driver breath using MQ3 alcohol gas sensor. If either function detects unsafe conditions, the main loop will trigger alert through LCD display and buzzer and send status updates to the mobile application via SIM card module.

For the device driver layer, it interacts directly with the system processor to control specific hardware components. The hardware components include the USB webcam, SIM card, MQ3 alcohol gas sensor, buzzer and LCD display.

4.2 System Components Specifications

4.2.1 Hardware

In hardware perspective, the main components involved in this project are Raspberry Pi 4 Model B, MQ3 alcohol gas sensor, USB webcam, MCP 3008 Analog to Digital Converter (ADC), LCD 1602, active buzzer and Redmi powerbank. Below is the specification for each component.

Raspberry Pi 4 Model B

Raspberry Pi is the main processing unit of the system. It runs OS, reads sensors, controls drivers and communicate with mobile app.

Figure 4.2.1.1 Raspberry Pi 4 Model B

Table 4.2.1.1 Raspberry Pi 4 Model B specification [10]

Description	Specification
Model	Raspberry Pi 4 Model B
Processor	Broadcom BCM2711
No. of core	Quad core
Core type	Cortex-A72 (ARM v8) 64-bit SoC @ 1.8GHz
Memory	2GB LPDDR4-3200 SDRAM
Wireless	2.4 GHz and 5.0 GHz IEEE 802.11ac, Bluetooth 5.0, BLE
Ethernet	Gigabit Ethernet
USB Ports	2 x USB 3.0, 2 x USB 2.0
GPIO	40 pin GPIO header
Display Ports	2 × micro-HDMI® ports (up to 4Kp60 supported)
Camera Ports	2-lane MIPI CSI
Display Ports	2-lane MIPI DSI

Video Decode/Encode	H.265 (4Kp60 decode), H.264 (1080p60 decode, 1080p30 encode)
Graphics	OpenGL ES 3.1, Vulkan 1.0
Audio	4-pole stereo audio and composite video port
Storage	Micro-SD card slot for loading OS and data storage
Operating System (OS)	Raspbian (Raspberry Pi OS)
Technical condition	0 - 50 °C ambient
Power (USB-C)	5V DC via USB-C connector (minimum 3A*)
Power (GPIO)	5V DC via GPIO header (minimum 3A*)
Power over Ethernet	PoE enabled (requires separate PoE HAT)

MQ3 Alcohol Gas Sensor

This sensor detect alcohol vapor in driver breath and measure in voltage.

Figure 4.2.1.2 MQ3 Alcohol Gas Sensor

Table 4.2.1.2 MQ3 alcohol gas sensor specification [11]

Description	Specification
Power Supply	5VDC (@165mA heater ON / 60mA heater OFF)
Current	150 mA
Digital Output (Do)	0 and 1 TTL digital (0.1V and 5V)
Analog Output (Ao)	0.1V to 0.3V (pollution related), max concentration voltage: 4V
Alcohol Concentration Detection	0.05 mg/L to 10 mg/L
Interface	1 TTL compatible input (HSW) and 1 TTL compatible output (ALR)
Heater consumes	<750mW

Resistance of heater	33 ohms ±5%
Operating temperature	-10°C to 50°C (14°F to 122°F)
Storage temperature	20°C to 70°C
Load resistance	200k ohms
Sensitivity Rs	≥5: Rs(in the air) / Rs(0.4mg/L Alcohol)
Sensing resistance	1MΩ to 8MΩ @ 0.4mg/L alcohol
Sensor dimensions	32x22x16 mm
Humidity	<95%RH
Oxygen Concentration	21%
Slope rate	≤0.6
Preheating duration	20 seconds

Logitech C270 HD Webcam

The USB webcam is used to capture driver face in real time. The main functionality is to authenticate driver and detect drowsiness.

Figure 4.2.1.3 Logitech C270 HD Webcam

Table 4.2.1.3 Logitech C270 HD Webcam specification [12]

Description	Specification
Height	72.91 mm
Width	31.91 mm
Depth	66.64 mm
Cable length	1.5 m

Weight	75 g
Max Resolution	720p /30fps
Camera mega pixel	0.9
Lens type	plastic
Focus type	Fixed focus
Built-in mic	mono
Mic ranges	Up to 1m
Diagonal field of view (dFoV)	55° Universal mounting clip fits laptops, LCD or monitors

MCP3008 ADC

This ADC works with MQ3 alcohol sensor. As the alcohol sensor produce analog output, thus it need to work with ADC to convert the analog output to digital. After conversion, the data produced by the sensor is compared with the threshold value.

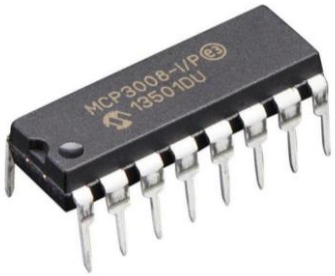


Figure 4.2.1.4 MCP3008 ADC

Table 4.2.1.4 MCP3008 specification [13]

Description	Specification
Resolution	10-bit
Input Channels	8 (MCP3008) / 4 (MCP3004 variant)
Input Configuration	Programmable as single-ended or pseudo-differential pairs
Sample-and-Hold	On-chip sample and hold
Interface	SPI serial interface (modes 0,0 and 1,1)
Power Supply Voltage (VDD)	2.7 V – 5.5 V
Max Sampling Rate	200 ksps (at VDD = 5 V), 75 ksps (at VDD = 2.7 V)
Standby Current	5 nA typical, 2 µA maximum

Active Current	500 µA maximum at 5 V
Technology	Low Power CMOS
Operating Temperature	-40 °C to +85 °C (industrial range)

Active Buzzer

This active buzzer is used for alert purpose. It triggered when unsafe conditions are found during drowsiness detection and alcohol detection step.

Figure 4.2.1.5 Active Buzzer

Table 4.2.1.5 Active Buzzer specification [14]

Description	Specification
Operating Voltage	5 V
Operating Current	15-30 mA
Sound Output	85 dB
Resonant	4 kHz
Duty Cycle	1.5 ~ 2.5 kHz
Size	max 50% duty
Operating Temperature	-20°C to +70°C

LCD 1602

The LCD display is used to display the system status including the authentication result, drowsiness detected, and alcohol detected.

Figure 4.2.1.6 LCD 1602

Table 4.2.1.6 LCD 1602 specification [15]

Description	Specification
I2C Address	0x27
Backlight	Black character on yellow background
Supply voltage	5V
Size	2x35x18 mm
Interface	Come with I2C interface

Redmi Power bank

The power bank is used to power the Raspberry Pi. Since the prototype is designed to be deployed in real-world driving scenario, using power bank can offer mobility and convenience compared to an adapter.

Figure 4.2.1.7 Redmi Power Bank

Table 4.2.1.7 Redmi Power Bank specification [16]

Description	Specification
Product model no.	PB200LZM
Battery type	Lithium-ion polymer battery
Battery power	74 Wh/3.7 V/20,000 mAh

Rated capacity	12,000 mAh (5.1 V/3.6 A)
Input ports	Micro-USB/USB-C
Output ports	2 × USB-A
Output parameters	5 V⎓2.1 A/9 V⎓2.1 A/12 V⎓1.5 A
Output parameters (single-port output)	5.1 V⎓2.4 A/9 V⎓Max. 2 A 12 V⎓Max. 1.5 A
Dual-port output	5.1 V⎓3.6 A
Product dimensions	154 × 73.6 × 27.3 mm

4.2.2 Software

Raspberry Pi OS

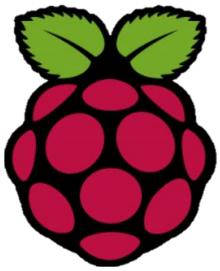


Figure 4.2.2.1 Raspberry Pi logo

Raspberry Pi OS, also known as Raspbian, is an official operating system specifically design for Raspberry Pi computer. It provide a desktop interface which allow user to write Python code that control the hardware interfaced on the board such as sensors, buzzer, camera, LEDs and GPIO pins. Other than that, user can edit the code via built in code editor such as Geany. Also, Raspberry Pi OS consist of command terminal that help user to execute instruction and debug the code easily.

Flutter

Figure 4.2.2.2 Flutter logo

Flutter is an open-source UI software development framework created by Google. It allows user to develop beautiful and interactive multi-platform applications using single codebase. The code is written in Dart programming language. In this project, Flutter is used to develop the driver monitoring interface which is able to display driver status and authenticate results received from the Raspberry Pi through Wi-Fi. Also, many other interesting features and widgets are integrated to make the mobile app more interesting and user friendly.

Raspberry Pi Imager

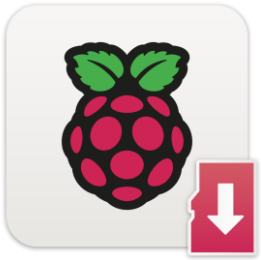


Figure 4.2.2.3 Raspberry Pi Imager logo

Raspberry Pi imager is an application that provide alternative solution for user to install Raspberry Pi OS onto SD card. In this project, this application is used during the setup phase to install Raspberry Pi OS onto the SD card. User is allowed to customize the settings before the first boot such as configure the network settings like hostname, Wi-Fi and password. This step is important as the whole project is deployed and managed remotely via secure shell (SSH) and virtual network computing (VNC).

RealVNC Viewer

Figure 4.2.2.4 RealVNC logo

RealVNC is a remote desktop software that allow users to access and control Raspberry Pi from another device over a network connection. To use this software, Raspberry Pi need to connect to the same Wi-Fi as the device. Upon connect, it provide graphical user interface (GUI) to Raspberry Pi, and user can interact without the need of external monitor, keyboard or mouse. In this project, RealVNC is used alongside SSH to provide remote connection. All the project algorithms are deployed in this virtual environment for coding, testing and debugging.

Google Colab

Figure 4.2.2.5 Google Colab logo

Google Colab is a cloud-based platform powered by Google that allow user to execute and debug Python code in web browser. It is widely used for machine learning as it provide free access to GPU, shared storage and preconfigured libraries. In this project, Google Colab was used to build face authentication model and train eye-closed detection model.

4.3 Circuits and Components Design

Raspberry Pi

Raspberry Pi 4 Model B has 40-pin GPIO header that provides power, ground, and I/O connections. The power pins include 3.3V and 5V, multiple ground pins and GPIO pins that are programmable to read sensor and control hardware components. Some GPIO pins has specialized functions like SDA and SCL pins are used for I2C communication and UART pins for serial communication.

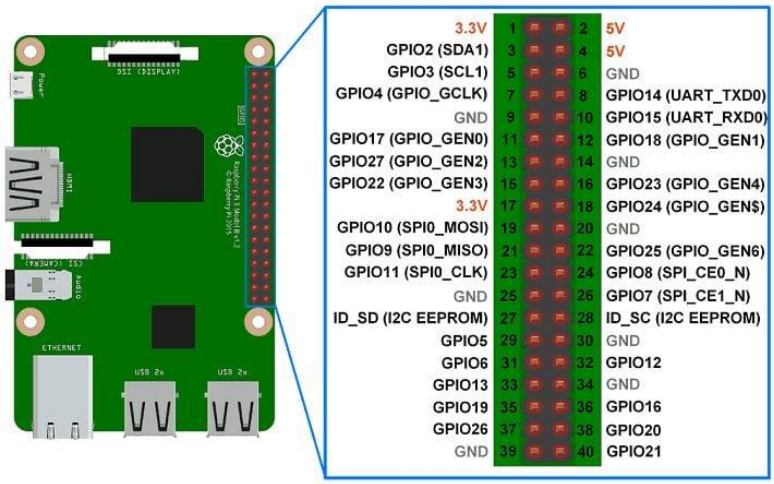


Figure 4.3.1 Raspberry Pi pin layout

MQ3 Sensor Connection

The MQ3 alcohol sensor produces an analog voltage output (AO) that varies with the concentration of alcohol vapor in the surrounding air. However, Raspberry Pi is not capable to read the analog signals directly, therefore an ADC is required to convert the analog signal to digital signals. In this case, the AO pin of sensor is connected to the MCP3008 ADC. After conversion, Raspberry Pi will determine whether the alcohol concentration exceeds the predefined threshold value.

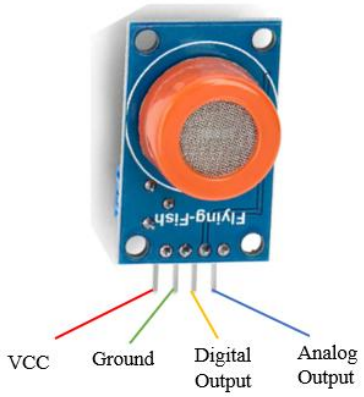


Figure 4.3.2 MQ3 sensor pin layout

Table 4.3.1 MQ3 sensor pin layout

Pin	Description
VCC	Power supply pin, connect to 5V pin
GND	Ground pin
AO	Analog output channel, used to receive data from ADC. This pin is connected to the MCP3008 ADC output pin.

MCP3008 ADC Connection

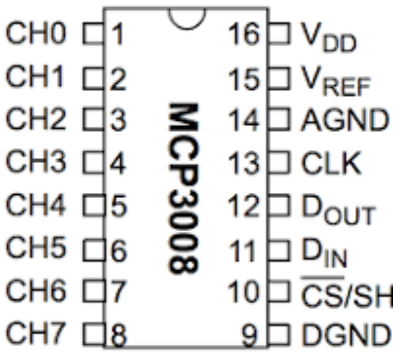


Figure 4.3.3 MCP3008 sensor pin layout

Table 4.3.2 Pin connection between Raspberry Pi and MCP3008

Raspberry Pi	MCP3008
Pin 1 (3.3V)	Pin 16 (V_{DD})
Pin 1 (3.3V)	Pin 15 (V_{REF})
Pin 6 (GND)	Pin 14 (AGND)

Pin 23 (SCLK)	Pin 13 (CLK)
Pin 21 (MISO)	Pin 12 (D_{OUT})
Pin 19 (MOSI)	Pin 11 (D_{IN})
Pin 24 (CE0)	Pin 10 (CS/SHDN)
Pin 6 (GND)	Pin 9 (DGND)

As mentioned earlier, MQ3 sensor need to integrate with ADC to convert the analog signal to digital signal so that Raspberry Pi can process the data. In this project, MCP3008 is used as it provides 10-bit resolution and up to eight analog input channels, which make it suitable for processing analog output from the MQ3 sensor. MCP3008 is powered with 3.3V on V_{DD} and V_{REF} to ensure it is compatible with Pi logic levels. The analog input from the MQ3 sensor is converted by MCP3008 and the digital value (ranging from 0 to 1023) is sent to Pi via SPI interface. Communication requires four core pins including MOSI, MISO, SCLK and CS, which are mapped to the Raspberry Pi's GPIOs pin accordingly. Once the ADC value is received, the corresponding voltage is calculated by using the formula:

$$Voltage = \frac{ADC\_Value}{1023} \times V_{REF} \quad (1)$$

where

ADC_Value are digital output from ADC

V_{REF} is the voltage supplied to ADC

After conversion, Pi is able to interpret the sensor output and compare it with the predefined threshold value. If the calculated concentration exceed the threshold, the buzzer is activated, and the status updates on the mobile app. Below is the connection diagram between Pi, MCP3008 ADC and MQ3 sensor.

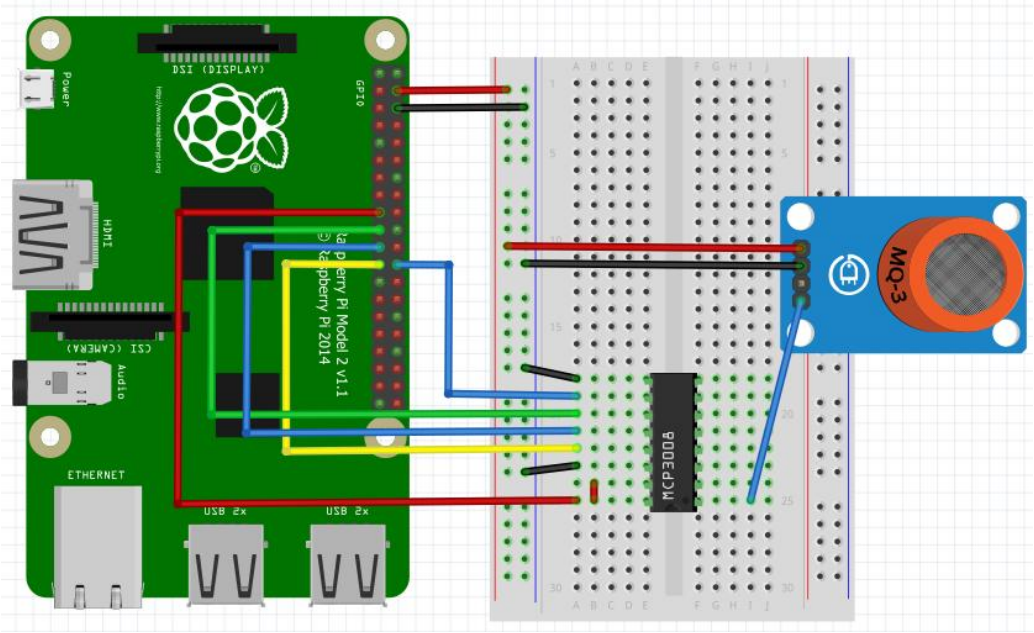


Figure 4.3.5 Connection of alcohol sensors and ADC

Active Buzzer Connection

The connection of active buzzer is relatively easy as it requires only digital control signal to operate. When the GPIO pin 18 is set to HIGH, current flows through the buzzer and producing sound.

Figure 4.3.5 Active buzzer pin layout

Table 4.3.3 Pin layout for active buzzer

Pin	Description
I/O pins	Connect to GPIO 18 / Pin 12
GND	Connect to GND pin

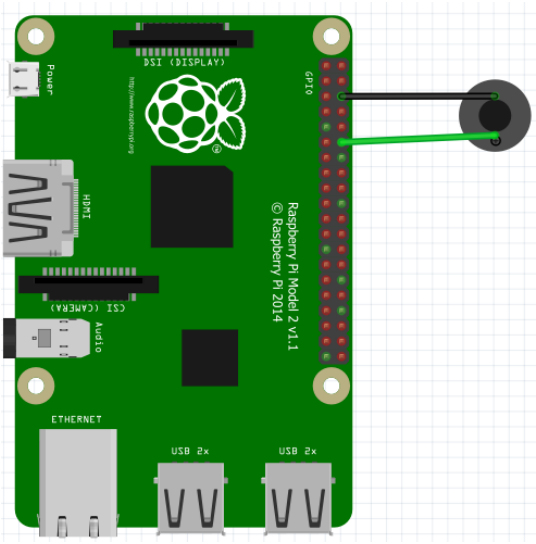


Figure 4.3.6 Connection of active buzzer

LCD Display Connection

The LCD display used has built in I2C which simplifies the wiring by using only four pins including GND, VCC, SCL and SDA. The communication is handled through I2C protocol, allowing for simple data transmission. The I2C address for the LCD is 0x27.

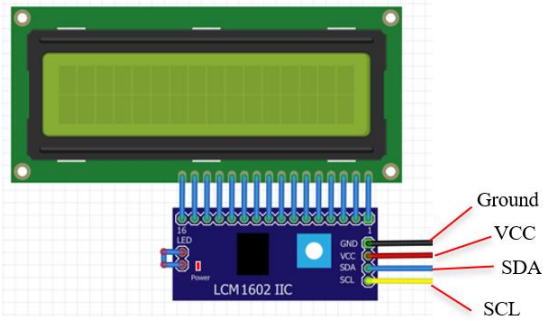


Figure 4.3.7 LCD 1602 with I2C pin layout

Table 4.3.4 Pin layout for LCD display with I2C

Pin	Description
GND (Ground)	Connect to GND pin
VCC (Voltage supply)	Connect to 5V pin
SCL (Serial Clock Line)	• Clock line for I2C communication. • Connect to SCL pin (GPIO Pin 3 / Pin 5)

SDA (Serial Data Line)	• Data line for I2C communication • Connect to SDA pin (GPIO 2 / Pin 3)

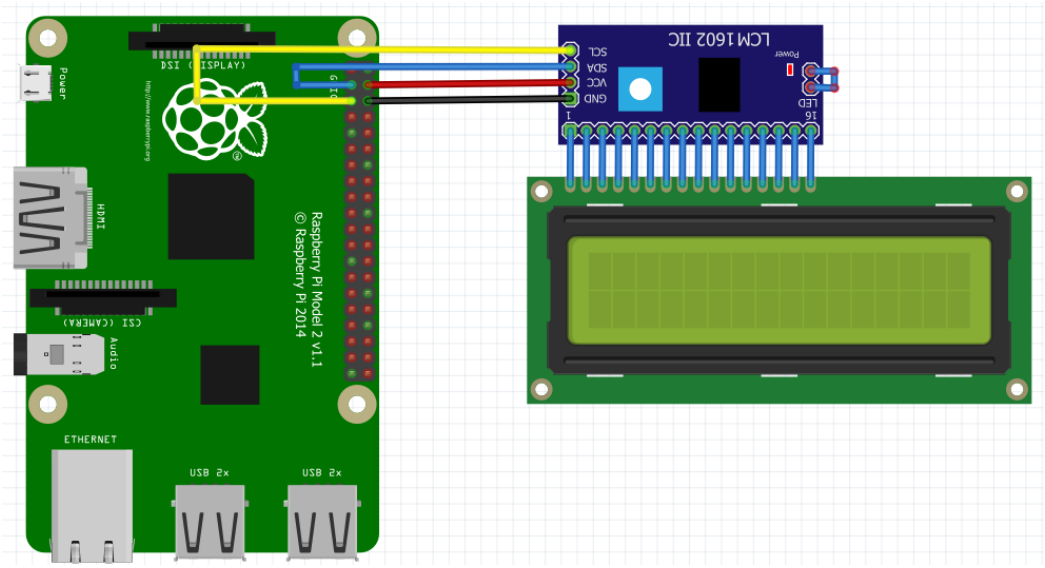


Figure 4.3.8 LCD 1602 with I2C connection

Complete Connection

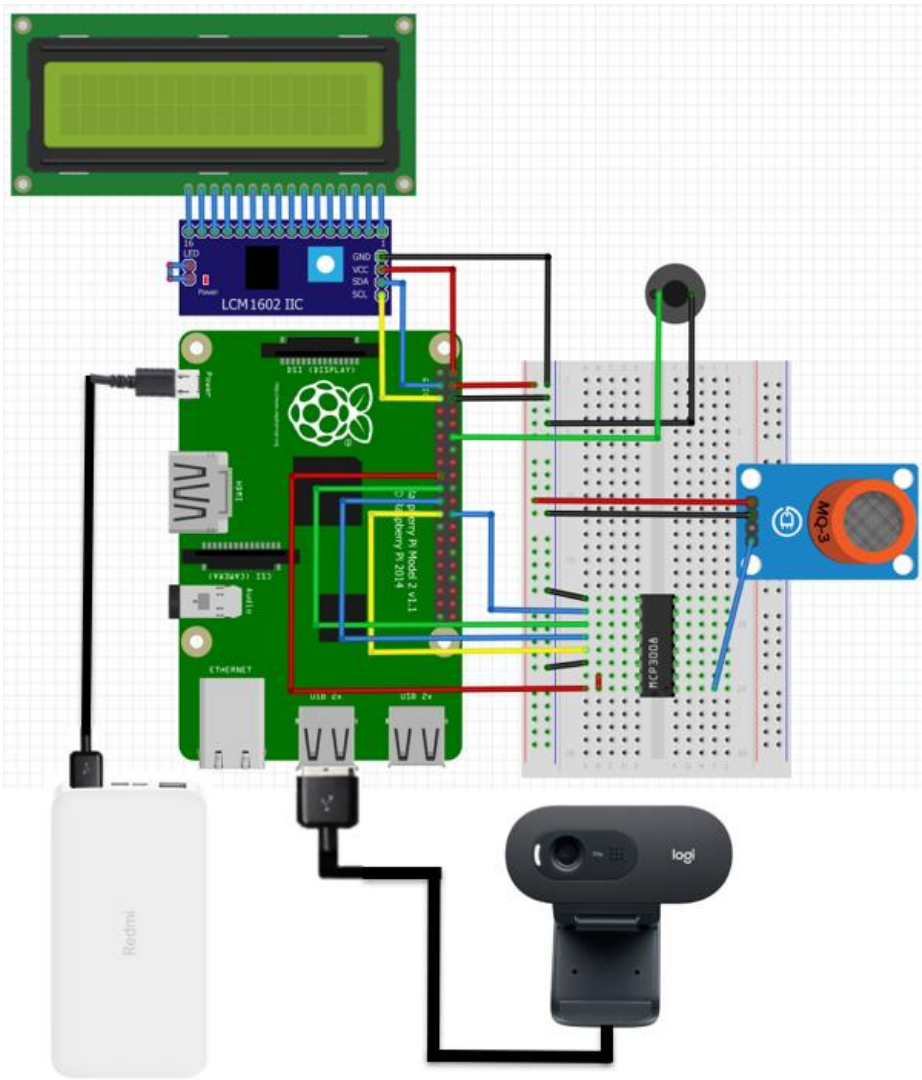


Figure 4.3.9 Complete connection of all components

4.4 Face Authentication Algorithm

This section will explain the concept behind the scenes of the face authentication algorithm including the method use, working mechanism and deployment.

4.4.1 Concept of Face Embeddings

In the proposed system, the method used for face authentication system is known as face embeddings. It is a vector of number that is encoded based on the unique features of a face image. The embeddings will extract out the important face information such as shape of eyes, nose, mouth, jawline, etc, and ignoring irrelevant information like lighting and background. Instead of simply comparing the similarity between raw images, this method convert each face into numeric representation in a feature space [17]. In other word, the similarity between two corresponding faces can be evaluated by the distance between two vectors.

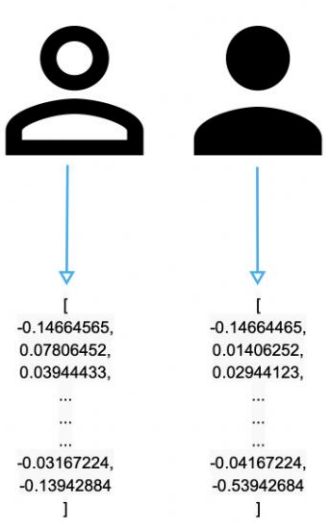


Figure 4.4.1.1 Face to vector embeddings

4.4.2 Why Face Embeddings Suitable for Face Authentication

The reason face embeddings are suitable to verify a person identity is due to every people have unique face features. Consider a scenario, where two images of same person are capture under different conditions like different lighting, angles or expression. If these two images are converted to vector embeddings and compare, then their vector must be close together. On the other hand, if the vector are compared with a totally different people, then the vector must be far apart.

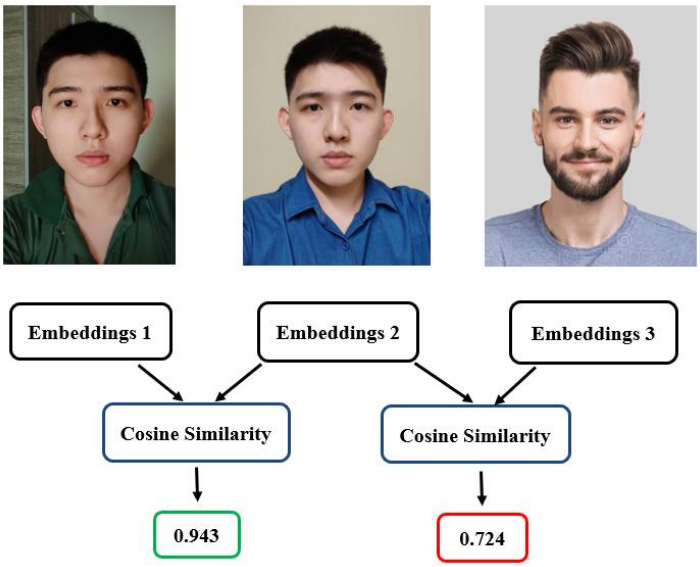


Figure 4.4.1.1 Comparison of embedding between different face

This method generalizes the idea of representation learning where the system adapts what features are important and encodes them into a more compact and machine-readable format. The advantage of this approach is that once embeddings are created, face authentication are more towards to vector similarity problem where can be solved by using mathematical distance measures (Cosine similarity). This make the system more efficient and faster than other pixel-based approach like CNN method.

4.4.3 Comparing Face Embeddings

The similarity between two vectors can be evaluated by using cosine similarity formula. The result range from 0 to 1. The closer the result to 1.0 indicates the vector are close to each other. On the other hand, the result far away from 1.0, indicate the vector are likely unrelated.

The formula to compare face embeddings as shown below.

$$cosine\_similarity = \frac{A \cdot B}{||A|| \cdot ||B||} \quad (2)$$

where

A – Vector embeddings of first face

B – Vector embeddings of second face

For example, assume we use the first three values

- Embedding of Face A = [0.12, -0.98, 0.64, ….]
- Embedding of Face B = [0.14, -0.68, 0.55, …]

Compute similarity:

$$cosine\_similarity = \frac{(0.12 \times 0.14) + (-0.98 \times -0.68) + (0.64 \times 0.55)}{\sqrt{(0.12^2 + -0.98^2 + 0.64^2)} \times \sqrt{(0.14^2 + -0.68^2 + 0.55^2)}}$$

$$= 0.993$$

Since the cosine similarity yield 0.993 which close to 1, thus we may conclude these two faces are likely the same person. However, the actual calculation is much more complex as the vector embeddings are longer (768-dimensional vector) with lengthy decimal point.

4.4.4 Generate Face Embeddings

A Convolutional Neural Network (CNN) is a type of deep learning model designed to process visual data by learning patterns from the data images. It works by applying multiple layers of convolutions and pooling to extract features of increasing complexity. At the beginning, an input face image is passed through convolutional layers where filters detect low-level patterns such as edges and corners. Pooling reduces the spatial size, keeping only the most important information mainly facial features and excluding other background noises. This making the model more robust to small variations.

As the image moves deeper into the network, more convolution and pooling layers are applied to capture higher-level features including shapes, textures and, more abstract structures like facial components. Finally, the extracted feature maps are flattened and passed through fully connected layers, which compress the information into a compact numerical vector known as face embedding. This vector encodes the semantic information of the face which can be used for face comparison purpose.

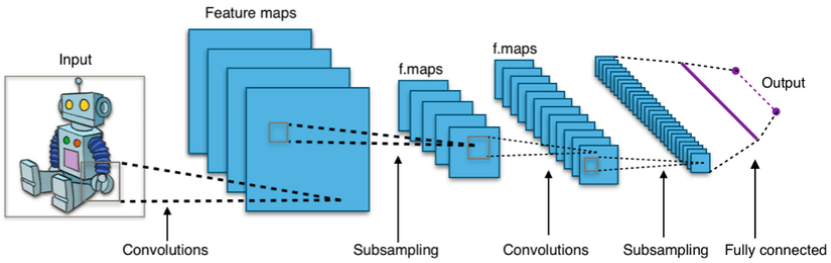


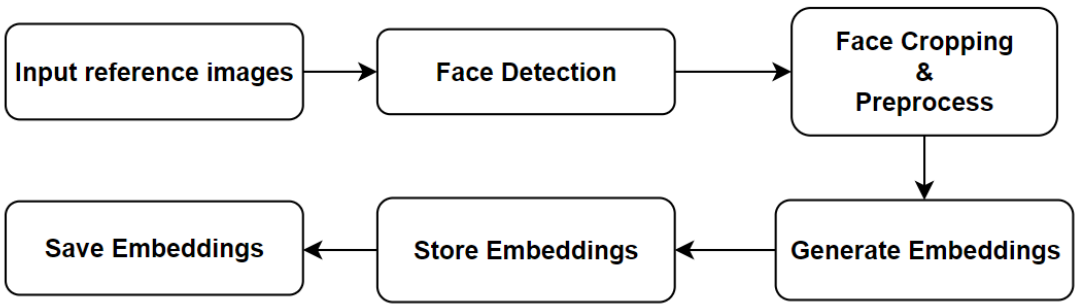
Figure 4.4.4.1 CNN architecture

The effectiveness of this embedding model relies heavily on the dataset used during training. A large and diverse labelled dataset, containing many variations of the same class across different conditions (such as lighting, pose, and background), allows the network to adjust its weights so that images with the same label are projected close together in the embedding space, while images with different labels are pushed further apart. Through this optimization, the network learns to encode only the essential discriminative features of the image, making the final embedding a powerful representation for similarity search, verification, or classification tasks.

In this project, the Python library imgbeddings applies the concept of feature extraction by leveraging a pretrained EfficientNet CNN, which has been trained on large-scale image datasets such as ImageNet. As we know, ImageNet contains over 14 million labelled images across thousands of object categories, enabling the network to learn general-purpose visual representations like edges, textures, and shapes. These learned features transfer well to new tasks, including faces, even though the model was not explicitly trained on face datasets. When a cropped face image is passed through the EfficientNet backbone, the network encodes it into a 768-dimensional embedding vector, capturing distinctive facial patterns and structures. This embedding can then be compared against stored embeddings for similarity-based face authentication.

4.4.5 Algorithm Flow of Face Authentication

User Image Database Preparation



The algorithm flow above show the preparation of database for the authorized user. Before the authentication begin, the face embeddings of the user need to be saved in the database beforehand as a reference for later comparison. The process begins with collecting the user images. Below is the dataset image of the authorized user.

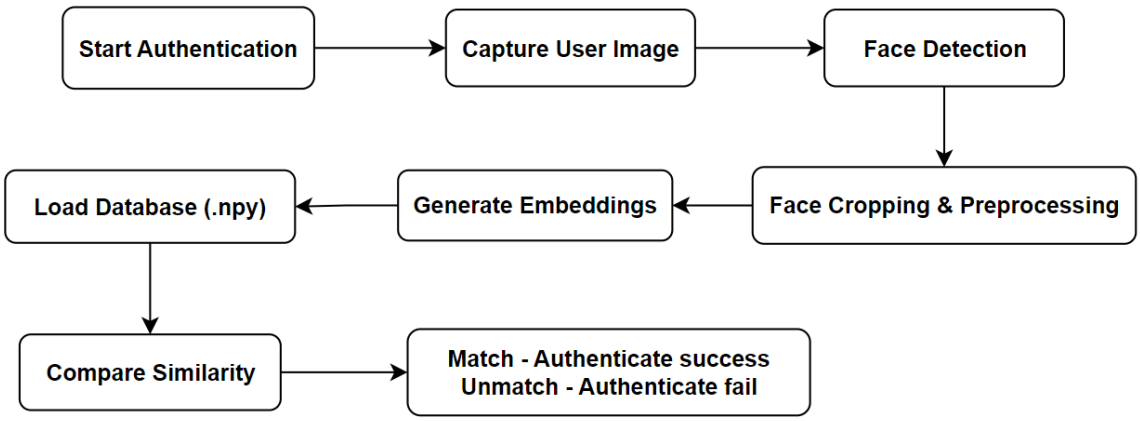
Figure 4.4.5.1 Sample reference images

Each image will undergo face detection to locate the face, followed by cropping and preprocessing. During image preprocessing, the image will run through RGB conversion, enhance sharpness, enhance contrast and adjust brightness. After enhancement, the face is then converted into an embedding vector using imgbeddings and stored in an array. Finally, all the vector embeddings are saved in a single .npy file. The face database for the user is now ready. Example of face embeddings store in .npy file.

```
Data:
[[-0.01902168 -0.46284324  0.09570944 ...  0.5700505   0.6832519   0.1661438 ]
 [-0.01784959 -0.533584    0.34578893 ...  0.38696975  0.73458385  0.3076474 ]
 [-0.23243614 -0.33541563  0.625058   ...  0.21653499  0.5221455   0.2790224 ]
 ...
 [-0.07193629 -0.5742757   0.14316295 ...  0.36257032  0.9317264   0.38868153]
 [-0.16517292 -0.67803234  0.41892648 ...  0.34913674  0.05840508  0.1702801 ]
 [-0.06749234 -0.6852512   0.28417337 ...  0.66303235  0.5683624   0.09106119]]
```

Figure 4.4.5.2 Sample vector embeddings generated in .npy file

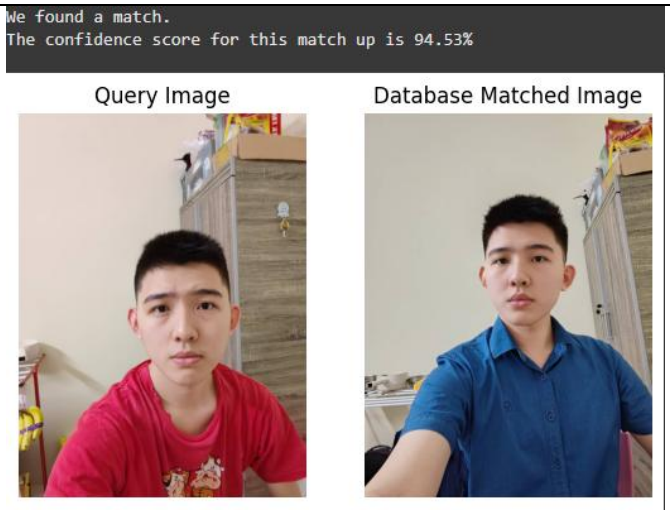
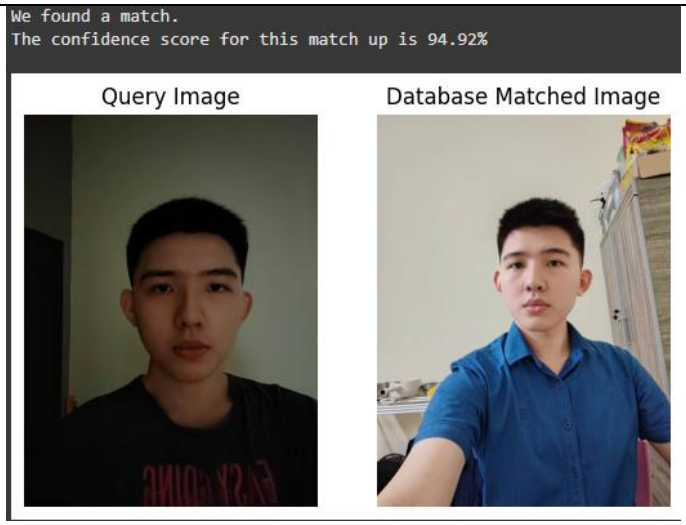
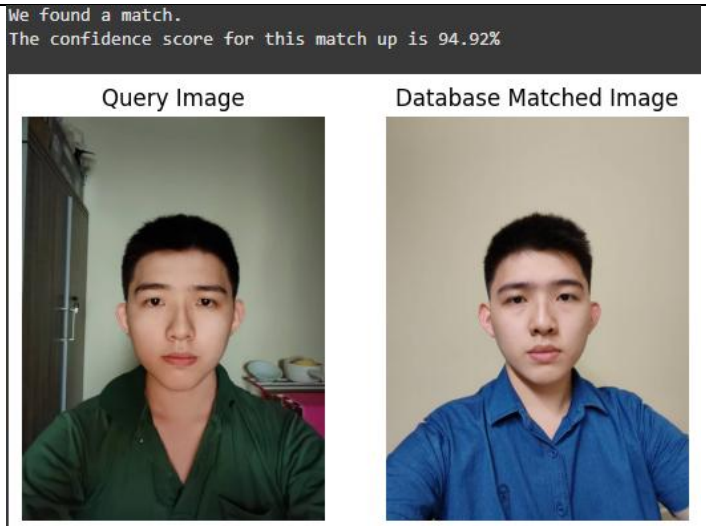
Face Authentication Algorithm Flow

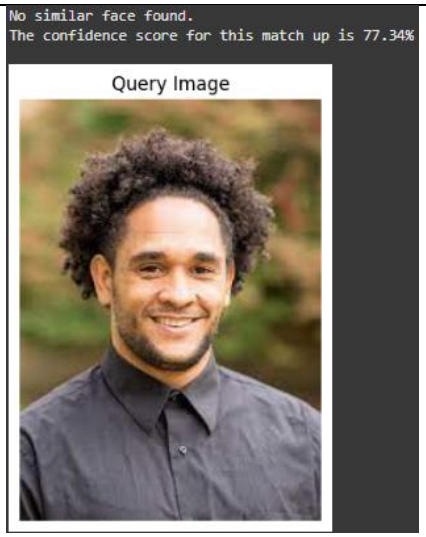
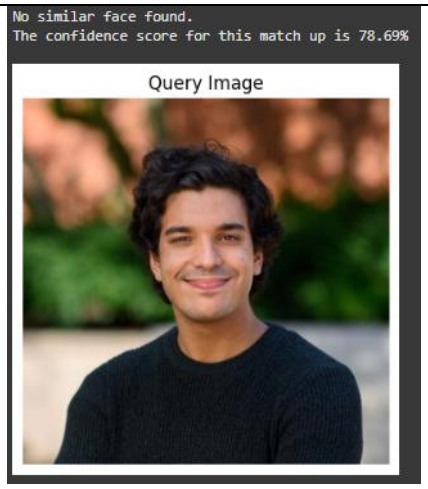


The above flow show the authentication flow when user using the mobile application. When the authentication start, the camera will start capture frame. When a face is detected, then the preprocessing steps including face cropping and image enhancement will take place. After that, the system will generate vector embeddings for the detected face and compare with the embeddings that store in the database using cosine similarity. If the confidence scores more than 90% and success for three consecutive times, then the user is granted access. inversely, if he confidence score < 90% and fail for three consecutive times, then the authentication fail.

Preliminary Test Result in Google Colab

The model is first deployed in Google Colab to evaluate its robustness. Several images were fed to check whether the face embeddings method able to classify the authenticate user. The criteria to pass is the confidence score must more than 90%. Below is several test case and the result.

No	Sample	Confidence (%)	Result (>90%)
1		94.53	Pass
2		94.92	Pass
3		94.92	Pass

4		79.12	Fail
5		77.34	Fail
6		78.69	Fail

By evaluating the result, it proven that face embeddings are indeed an ideal approach to authenticate face. The overall result are accurate where the model able to identify valid user correctly. For valid user (Image 1, 2 & 3), the system consistently achieved high confidence level, which score even higher than the predefined threshold (90%). While the confidence score

for other imager are less than the threshold. This result indicates that embeddings vector that map the facial features are accurate and able to differentiate different features.

4.5 Drowsiness Detection Algorithm

In this section, three drowsiness detection algorithm used in this project will be explained. This include eye landmark, mouth landmark and nose-chin landmark.

4.5.1 Face Landmark Detection

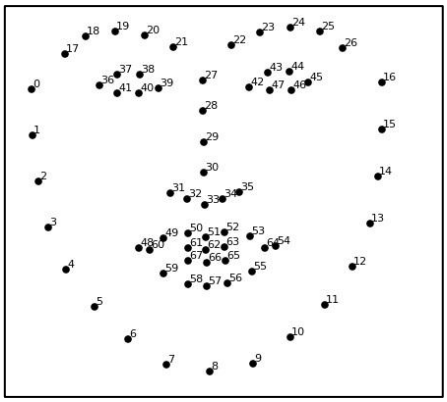


Figure 4.5.1.1 Face landmark

The drowsiness detection system begins with detecting driver's face by using dlib's 68- point facial landmark predictor. The predictor is a pre-trained machine learning that detect important point on the human face. When the camera captured human face, this predictor will map 68 specific landmarks on the detected face including facial regions, jawline, eyes, nose and mouth. To make variation on each facial features, the landmarks are indexed by a consistent value. In the proposed system, four facial features are taken note including eyes, mouth, nose tip and chin. Below is the landmark for each features.

Table 4.5.1.1 Landmark for eyes, mouth, nose and chin

Facial Features	Landmark
Right eye	36 – 41
Left eye	42 - 47
Mouth	48 - 59
Nose tip	33
Chin	8

4.5.2 Eye Landmark – EAR

The most reliable way to address drowsiness is through calculate EAR. EAR is a method used to indicate whether an eye is closed or not. It is computed by taking the average of vertical distances between eyelids and dividing it by horizontal width of the eye. The formula is given as follow.

$$EAR = \frac{\|p_2 - p_6\| + \|p_3 - p_5\|}{2 \cdot \|p_1 - p_4\|} \quad (3)$$

where

$p_1, p_2, p_3, p_4, p_5, p_6$ are six landmarks around eye.

$\|p_1 - p_4\|$ represent the horizontal distance, $\|p_2 - p_6\|$ and $\|p_3 - p_5\|$ represent the vertical distance. For example, the figure below show the eye landmark of an eye.

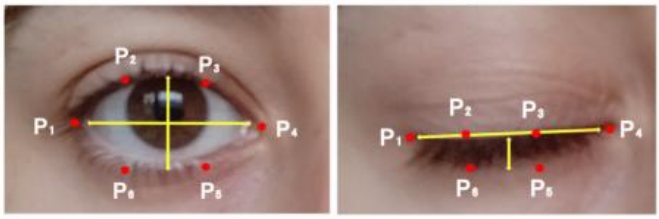


Figure 4.5.2.1 Eye landmark

By comparing these two figures, notice that, when the eye is opened, the EAR value increases. On the other hand, when the eye is closed, then the EAR value is decrease. In this case, by adding a threshold and counter, the EAR can be used to detect drowsiness.

In terms of threshold value, by considering every individuals will have varied eye landmarks, thus using a fixed threshold will be impractical. To address this problem a calibration state is introduced during face authentication phase. The reason why calibration is taking place during face authentication phase is because the driver will position their face well to the camera, so it is a good opportunity to set a threshold during this time. During calibration, the system will records EAR over several frames. The threshold is calculated by $EAR_{thres} = EAR_{avg} \times 0.93$. When enter drowsiness detection stage, when the EAR drop below EAR_{thres} for more than 1 second, it means the driver eyes are drowsy.

4.5.3 Mouth Landmark – MAR

Yawning is another sign of fatigue. To detect yawning, the system relies on twenty landmark point around the mouth. Given the formula to calculate MAR as follow:

$$MAR = \frac{\|p_1 - p_5\| + \|p_2 - p_4\|}{2 \cdot \|p_6 - p_3\|} \quad (4)$$

where

$p_1, p_2, p_3, p_4, p_5, p_6$ represent six landmarks around mouth

The concept of MAR is the same as EAR where the ratio is calculated by diving the vertical distance between the upper and lower lips by horizontal width of the mouth. In normal, when the driver is not yawning, the MAR is low. However, when yawning, the vertical mouth opens wides cause spike in MAR value.

The threshold set for MAR is 0.80 and a time condition is added to avoid misclassified. In experiment, when the MAR is more than 0.80 and more than 2 seconds, then the system will classify the event as yawning and trigger alert.

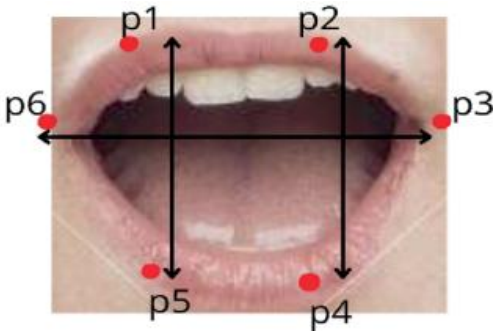


Figure 4.5.3.1 Mouth landmark

4.5.4 Nose-Chin Landmark - NLR

Other than eyes and mouth, head posture also can be used as a sign of drowsiness, especially when the drive begins to nod forward. To detect this, the system computes NLR using two facial landmarks including nose tip and the chin.

$$NlR = \frac{y_{chin} - y_{noseTip}}{ED} \quad (5)$$

where

y_{chin} is landmark of chin, 8

$y_{noseTip}$ is landmark of nose tip, 33

ED is horizontal distance between eyes

The ratio is calculated by measuring the distance between nose tip and chin, then normalize by the horizontal distance between eyes. When the driver sitting straight and look in front while driving, the ratio will remain stable. However, when the driver head is bending downward, then the ratio will decrease significantly. The threshold for NLR is similar to EAR, where it is set during calibration stage. During calibration, the NLR is recorded, and the threshold is set about 90% of this value. If the measured NLR drop below the threshold for more than 2.5 seconds, than the system concludes driver head is bent down and issue an alert.

In practical, head bend down not only indicates drowsiness but also indicates the driver is not paying attention while driving. For example, some drivers tend to look down on their mobile phone or are distracted. By using NLR, it can address this issue and alert the driver as well.

Figure 4.5.3.2 Head bend down when using phone

As a summary, this measurement not only to keep driver refresh but also ensure driver to stay alert while driving.

4.5.5 Results for Landmark Detection

Condition	Sample	Landmark value
Normal		EAR = 0.34 MAR = 0.53 NLR = 0.84
Eye Closed		EAR = 0.20 MAR = 0.54 NLR = 0.85
Yawning		EAR = 0.28 MAR = 1.26 NLR = 1.07
Head Bent		EAR = 0.11 MAR = 0.44 NLR = 0.56

4.6 Eye-Closed Detection Model

This section will explain the step to develop an eye-closed detection model including problem definition, data preparation, model selection, model training, model evaluation and model deployment.

4.6.1 Problem Definition

The motivation of developing an eye-closed detection model in this project is to build a more robust driver drowsiness detection to determines whether driver's eyes are closed. As mentioned in previous section, one of the method to classify whether a driver is sleepy is through calculating the EAR. In early stage, drowsiness was classified when the EAR value dropped below fixed threshold for 2 seconds. But during experiment, one major problem is noticed, which the EAR value is highly dynamic and fluctuates significantly across each camera frames. The reason why EAR is dynamic because it depends on facial landmark stability, head movement and lighting condition. If long duration is used like 2 seconds, many false negatives happen, which likely to reset the counter. In practice, this caused the EAR to occasionally rise above the threshold even when the eyes remained closed leading to misclassification and unreliable detection.

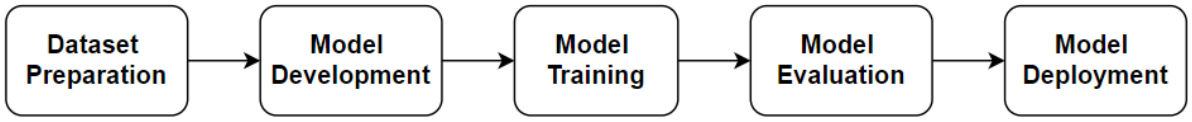
On the other hand, the implementation of deep learning–based eye-closure detection model can directly classify whether an eye is open or closed with high accuracy. This is because the model learns from visual features rather than relying on geometric thresholds. However, since the system need to operate on Raspberry Pi, deploying the model on every frame is not practical, as it is computationally expensive and can cause high CPU usage.

As a final solution, to balance between accuracy and efficiency, this project adopt hybrid approach in terms of eye closed detection where

- The EAR is monitored continuously because it is lightweight
- If the EAR drops below threshold for short duration (e.g. 0.7 seconds), the system activate the eye-closed detection model to classify whether the driver's eyes are truly closed.

By combining these two approach together, it can reduce false detection and less computation heavy. This strategy is suitable for real-time deployment on embedded devices like Raspberry Pi.

4.6.2 Model Preparation Algorithm Flow



Step 1: Dataset Preparation

In this project, the dataset used is obtained from Kaggle repositories. The original dataset consist of four classed including Open Eyes, Closed Eyes, Yawning and No-Yawning images. As the focus of our training model is to classify the driver eye status whether it is closed or opened, thus only Open Eyes and Closed Eyes dataset are considered. The total number of dataset is 1452 and the images are balanced for both eye state where each contains 726.

Figure 4.6.2.1 Sample open closed eye dataset

The dataset need to preprocess beforehand to ensure data consistency this include:

- **Resizing –** All images were resized to 32 × 32 pixels
- **Label encoding –** Since there are only two possible outputs (Closed or Open), the string labels converted to binary values where 0 = Closed and 1 = Open.
- **Data augmentation –** Random rotation, zoom, and horizontal flipping are applied to increase dataset diversity and improve generalization.
- **Train-test split –** The dataset was divided into 80% training and 20% testing.

(Train set size = 1161, Test set size = 291)

Step 2: Model Selection

A custom CNN model was designed using TensorFlow Keras. This is because the model need to deploy in Raspberry Pi which is a resource-constrained device with limited CPU capacity and no dedicated GPU. In this case, running heavy and computation-intensive models such as YOLO on Raspberry Pi is impractical because these models are specifically designed for large-scale object detection that demand high processing power. In contrast, TensorFlow Keras provides more flexible, lightweight, and computational efficient solution. It can be used to design compact CNN architecture that is suitable for binary classification problem like eye-open versus eye-closed states. Another benefit of using Keras is it allow the model to convert into TensorFlow Lite (TFLite). TFLite is the optimized version of model that reduce model size which enable real-time monitoring while maintaining high accuracy.

For the neural network architecture used in building the model, it consist of three layers of data which include input later, hidden layer and output layer.

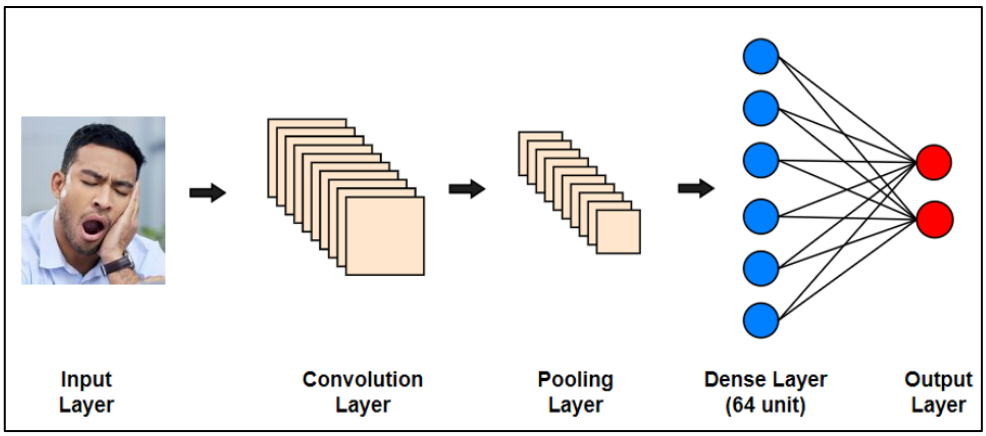


Figure 4.6.2.2 Network architecture for eye-closed detection model

- **Layer 1: Input Layer**

  The model process eye images of size 32 x 32 x 3 as input, where 32 x 32 is spatial resolution and 3 represent RGB channels. This step preprocess the image and feed to the network for further processing.

- **Layer 2: Convolutional Layers**

  The convolutional layers apply three filters including 16, 32 and 64 to the input image to learn on detect important spatial features of the eye image. This include the edges, textures and pattern of the eye region. Also, each layer is followed by Batch Normalization to stabilize the learning and improve the convergence speed.

- **Layer 3: Pooling Layers**

  After each convolutional block, a MaxPooling (2 x 2) layer is applied to reduce spatial dimensions of the feature maps while perceiving important information. Applying pooling can prevent overfitting problem to ensure only most significant features are passed to the next layer.

- **Layer 4: Hidden Layer (Dense)**

  The feature maps are then flattened into 1D vector and pass to fully connected Dense layer consist of 64 neurons. The activation function used is ReLU activation which extract useful non-linear features efficiently and effectively. The hidden layer serve as decision making stage by combining extracted features to learn complex patterns that distinguish between open and closed eyes. A dropout rate of 40% is applied to improve generalization and reduce overfitting by random ignoring some neuron during training.

- **Layer 5: Output Layer**

  The output layer consist only 2 neurons indicates it is a binary classification problem. There are only two outcome of the prediction which are either Open or Closed. Softmax activation function is used to ensure the output probabilities sum to 1. The class with higher probability is chosen as the final prediction.

Step 3: Model Training

The model was trained on labeled eye images using the Adam optimizer with a learning rate of 0.0005 and categorical cross-entropy loss. Training was performed in batches over multiple epochs, with dropout applied to reduce overfitting. During this process, the model iteratively adjusted its weights through backpropagation to minimize the loss and improve accuracy. The

training and validation performance were monitored to ensure the model was learning effectively without underfitting or overfitting.

Step 4: Model Evaluation

After the training was done, the model' performance was evaluated using validation dataset with accuracy as the main metric. Evaluation is an important phase to ensure the model was able to correctly classify whether eyes are open or closed. In this case, a confusion matrix was and several metrices including accuracy, precision, error rate and recall were calculated by using the formula below. Lastly, the loss and accuracy of training and testing set were displayed.

$$\textbf{Accuracy} = \frac{\mathrm{TP} + \mathrm{TN}}{\mathrm{TP} + \mathrm{TN} + \mathrm{FN} + \mathrm{FP}} \quad (6)$$

$$\textbf{Error Rate} = \frac{\mathrm{FP} + \mathrm{FN}}{\mathrm{FP} + \mathrm{FN} + \mathrm{TN} + \mathrm{TP}} \quad (7)$$

$$\textbf{Recall} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}} \quad (8)$$

$$\textbf{Precision} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP}} \quad (9)$$

where

TN – True Negatives

FP – False Positives

FN – False Negatives

TP – True Positives

Also, to validate the model robustness, several test cases were conducted by tuning the below hyperparameters:

- **Epochs -** One complete pass throughout the entire training dataset

- **Batch Size -** Number of trainings used in one forward and backward pass
- **Learning Rate -** How quickly the model learns during training
- **Number of Hidden Layer -** Layer between input and output later

Table 4.6.2.1 Baseline hyperparameter

Hyperparameter	Value
Epochs	20
Batch Size	16
Learning Rate	0.01
No. of Hidden Layer	1 (64 unit)

Table 4.6.2.2 Performance evaluation after turning the hyperparameter

Hyperparameter	Case	Testing Accuracy (%)
Epochs	5	90.03
	100	98.97
Batch Size	32	97.25
	128	97.25
Learning Rate	0.0001	94.85
	0.0005	97.56
Number of Hidden Layer	1 (64 unit)	97.25
	2 (64, 32)	92.10
	3 (64, 32, 16)	95.19

By evaluating the training accuracy and testing accuracy for each hyperparameter, the hyperparameters which obtained higher training and test accuracy are chosen. The chosen hyperparameter are set as shown in table below:

Table 4.6.2.3 Final hyperparameter

Hyperparameter	Value
Epochs	100
Batch Size	32
Learning Rate	0.0005
No. of Hidden Layer	1 (64 unit)

After the training was completed, the result are shown below.

Table 4.6.2.4 Final result

Metrices	Open Eyes Result (%)	Closed Eyes Result (%)
Accuracy	98.97	
Precision	97.95	100
Error Rate	12.78	97.95
Recall	100	98.0

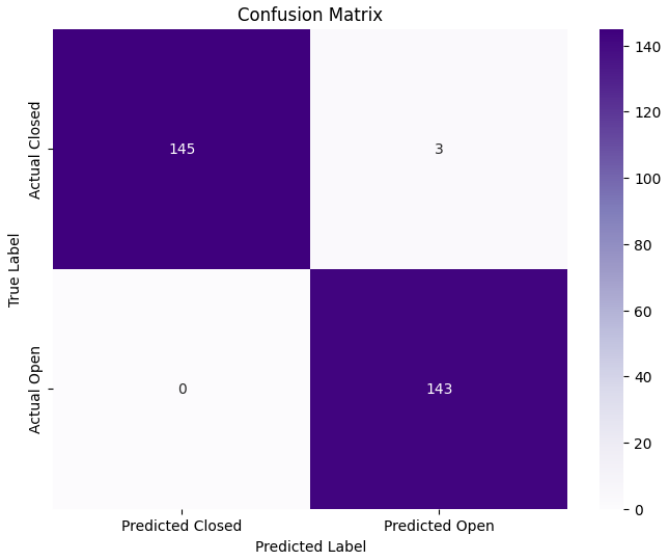


Figure 4.6.2.2 Confusion matrix

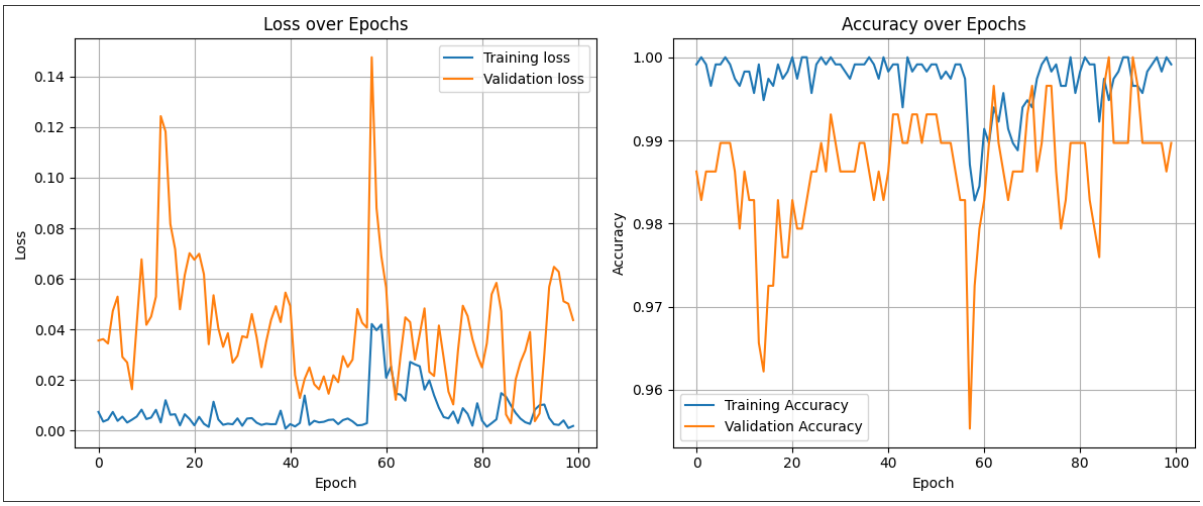


Figure 4.6.2.2 Train test loss and accuracy plot

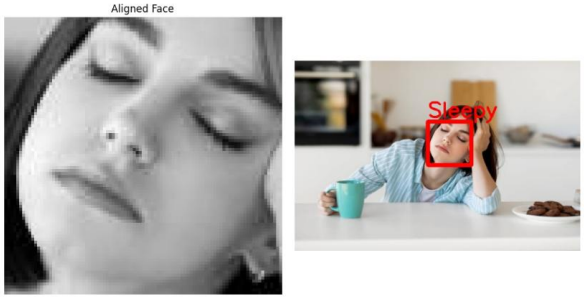
As a result, the confusion matrix demonstrate that eye state classification model performs extremely well, achieving overall accuracy nearly 99%. Out of 148 closed-eye images, the model correctly identified 145, with only three misclassifies as open. For open-eye samples, the model achieved perfect recognition, where it correctly classifying all 143 open-eyes samples without any mistake. This indicate that the model is highly robust and reliable and have very small tendency to mis-predict closed eyes as open.

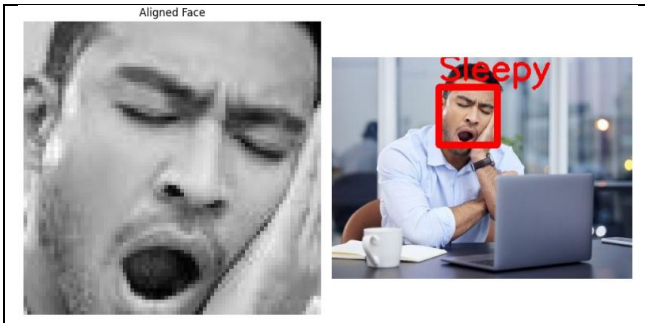
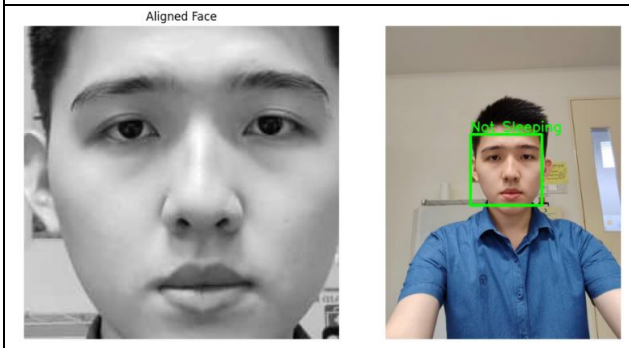
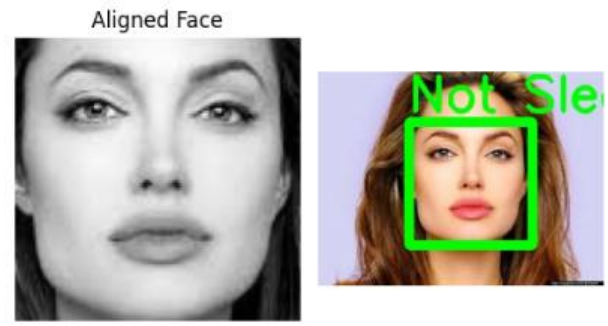
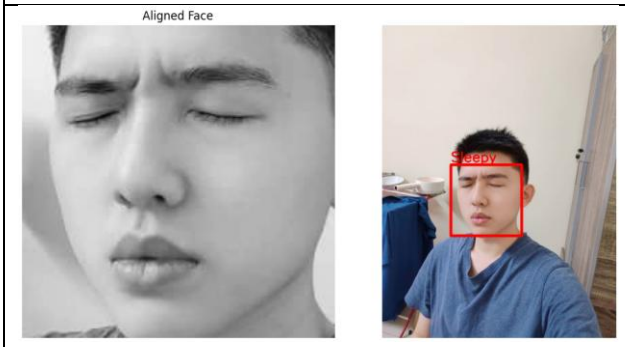
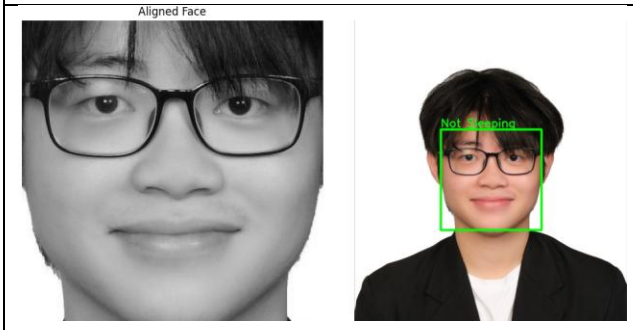
Step 6: Model Deployment

Model deployment is the final stage, where the evaluation on model will take place. The trained model was saved as .h5 file before deployment. The eye prediction flow are shown below:

1. **Face detection** – using dlib frontal face detector library to locate face
2. **Eye detection** – using Haar cascade classifier applied on the face region to locate eye position
3. **Preprocessing** – each detected eye was cropped, resized to 32 x 32, and passed to the CNN model
4. **Prediction** – the CNN will classify whether eye is Open (1) or Closed (0)
5. **Output** – if both eyes were classified as Closed, the driver was labeled as "Sleepy" with red bounding box applied. Otherwise, the driver was labeled as "Not Sleeping" with green bounding box applied.

Results:

Test Case	Expected Result	Predicted Result	Summary
	Closed Eye	Sleepy	Pass

	Closed Eye	Sleepy	Pass
	Open Eye	Not Sleeping	Pass
	Open Eye	Not Sleeping	Pass
	Closed Eye	Sleepy	Pass
	Open Eye	Not Sleeping	Pass

Based on the results obtained, the model is able to classify correctly for all test case image and ready to deploy in Raspberry Pi. Since the model will run on Raspberry Pi, the deployment must consider limited computational resources. Thus, the trained CNN model was exported in a lightweight format. In this case, the model is saved in TFlite format.

Chapter 5
System Implementation

5.1 Hardware Setup

The hardware setup for drowsiness detection system was designed to be suitable for deployment inside a vehicle. To achieve this, the system components were first arranged and tested individually, then placed all the components into a self-designed enclosure. The hardware setup is illustrated through four stages:

Stage 1: Enclosure Design in Tinkercad

The enclosure was initially designed in Tinkercad. The design enclosure is not just simply put all components inside, instead there are openings for ventilation, wiring, and hardware placement. This measurement is important to ensure all the hardware component to mount properly. For example, the two holes on the block were specifically designed for alcohol sensor and buzzer to ensure they are working effectively. The measurement for the enclosure including the length, width and height were carefully considered to ensure all the components fit well and in appropriate size to deploy in car.

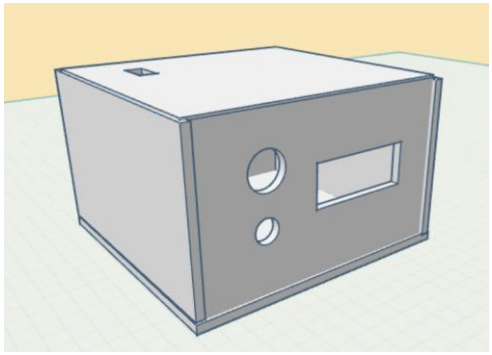


Figure 5.1.1 Tinkercad 3D enclosure design

Stage 2: Physical Enclosure

The enclosure design was fabricated using 3D printer. The 3D-printed enclosure provided durable and customization for the system. It ensure the components are not easily damaged and easier to place in vehicle environment.

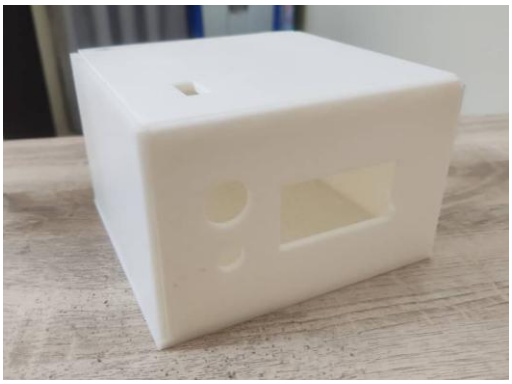


Figure 5.1.2 Prototype enclosure

Stage 3: Components Wiring

All hardware components were connected properly to the Raspberry Pi as describe in Chapter 4. The webcam was attached through USB, MQ3 alcohol sensor interfaced with MCP3008 through SPI pins, buzzer wired to GPIO pin 18, and LCD was connected through I2C. Each component are tested to ensure it is functional. The wiring is arranged properly so that each component can interfaced to the enclosure easily.

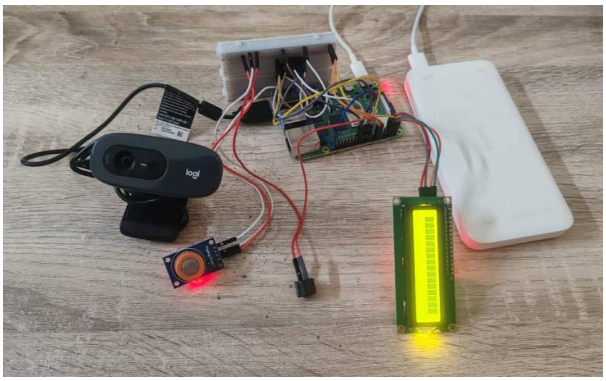


Figure 5.1.3 Components wiring

Stage 4: Final Prototype

The components were then placed inside the 3D-printed enclosure. The final prototype was compact where the Raspberry Pi and sensors were securely fixed, and the LCD and buzzer were accessible externally. The final prototype demonstrated a practical and reliable design, and it is ready for deployment.

Figure 5.1.4 Final prototype

5.2 Software Setup

Before deploying the code. there are several configurations that needed to be done beforehand. First, in command prompt, update and upgrade the latest packages and version with following command.

```
raspipi@jie:~ $ sudo apt update && sudo apt upgrade
Hit:1 http://deb.debian.org/debian bookworm InRelease
Get:2 http://deb.debian.org/debian-security bookworm-security InRelease [48.0 kB
]
Get:3 http://deb.debian.org/debian bookworm-updates InRelease [55.4 kB]
Get:4 http://deb.debian.org/debian-security bookworm-security/main armhf Package
s [239 kB]
```

Figure 5.2.1 Update and upgrade package

Next, as some libraries are not allowed to install directly in a global environment. Therefore, virtual environment is created to install it. After creating the virtual environment (myenv), run it with the following command to activate it.

```
raspipi@jie:~ $ source myenv/bin/activate
(myenv) raspipi@jie:~ $
```

Figure 5.2.2 Activate virtual environment

Then, all the necessary libraries that were used in our program were installed.

```
pip install opencv-python
pip install dlib
pip install numpy
pip install pillow
pip install flask
pip install flask-cors
pip install scipy
pip install imgbeddings
pip install tflite-runtime
pip install RPLCD
```

5.3 Mobile Application Setup

The mobile application is developed using Flutter. This app communicates with Raspberry Pi via REST API provided by the Flask server running on the PI. In the setup, Raspberry Pi and mobile device need to connect to the same Wi-Fi network. After that, to allow remote control and timely update from Raspberry Pi, the IP address of Pi is configured in the mobile application code. The IP address of Pi can be obtained as follow

```
(env) raspberi@chee:~/py-spidev-master $ hostname -I
192.168.1.101 2001:d08:e0:3c6b:6ba6:49ec:9aed:d71d
```

Figure 5.3.1 Check IP address

```
TextButton(
  onPressed: () async {
    Restart.restartApp();
    Navigator.pop(context); // Close the dialog first
    final piIp = "192.168.1.101:5000";
    try {
      final url = Uri.parse("http://$piIp/shutdown_pi");
      final res = await http.post(url);
```

Figure 5.3.2 Use Raspberry Pi IP address in Flutter code

5.4 Setting and Configuration

5.4.1 VNC configuration

The Raspberry Pi program was run on VNC server. On Raspberry Pi OS, VNC server is preinstalled but disable by default. Therefore, in configuration interface, enabled the VNC option. Then, run the remote desktop software, for example RealVNC Viewer to remote access Raspberry Pi.

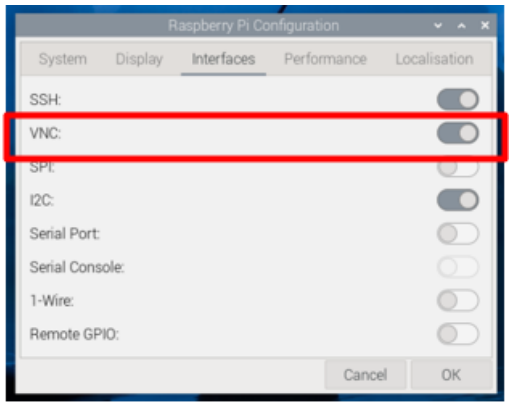


Figure 5.4.1.1 Enable VNC

5.4.2 LCD configuration

The LCD display used have built in I2C. Thus, to use alongside with Raspberry Pi, several configuration need to take part. First, in Raspberry Pi OS, enable I2C in configuration interface and reboot Pi to load the changes.

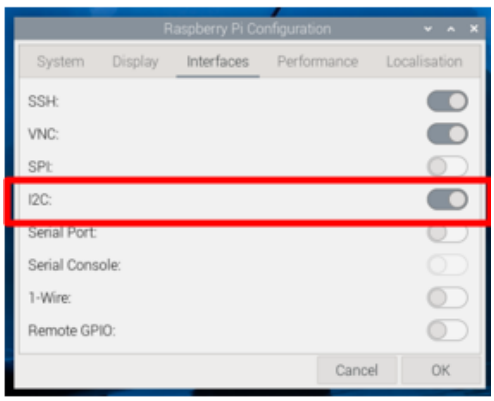


Figure 5.4.2.1 Enable I2C

After that, use command sudo i2cdetect -y 1 to check if Raspberry Pi detect the I2C signal. As shown as figure below, 0 x 27 is the I2C address of the LCD display. This indicates LCD display is doing fine.

Figure 5.4.2.2 I2C address

5.4.3 MCP3008 setup

MCP3008 is a 10-bit SDC that communicates with Raspberry Pi over SPI interface. First, enable SPI in configuration interface and reboot Pi to load the changes.

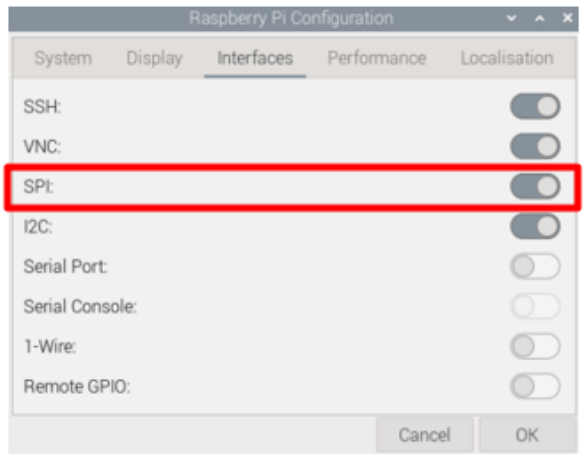


Figure 5.4.3.1 Enable SPI

Install Python spidev library to allow the communication with MCP3008 with following command.

```
wget https://github.com/doceme/py-spidev/archive/master.zip
unzip master.zip
cd py-spidev-master
sudo python setup.py install
```

After that, a small Python class MCP3008.py is created to send and receive data over SPI. Then, import this file to the main project folder with command.

```
from MCP3008 import MCP3008
```

Lastly, before use initialize the class with Equation (2) and the ADC is ready to use.

5.5 System Operation

After all the hardware and software are well setup, and the system is done configured, the drowsiness detection system is ready to deploy. At the beginning, Raspberry Pi runs the main Python program, which initializes all the hardware including camera, sensors, LCD and buzzer. The system then waits for a startup command from mobile application over the Flask REST API and waiting the startup command from mobile application.

```
[INFO] Loaded 5 embeddings.
INFO: Created TensorFlow Lite XNNPACK delegate for CPU.
 * Serving Flask app 'final'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment.
 Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://192.168.1.101:5000
Press CTRL+C to quit
```

Figure 5.5.1 Pi waiting for startup command

In mobile application view, the application begin with a welcome page. This mobile application is given name as “AutoSense”.

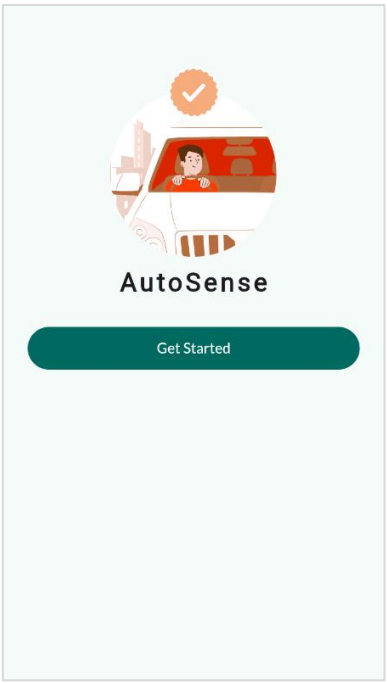


Figure 5.5.2 Welcome Page

When user press the “Get Started” button, it navigate user to the face authorization page. The “Start Face Recognition” is the startup command. When user press on it, Raspberry Pi will receive the start authentication command and activate the webcam.

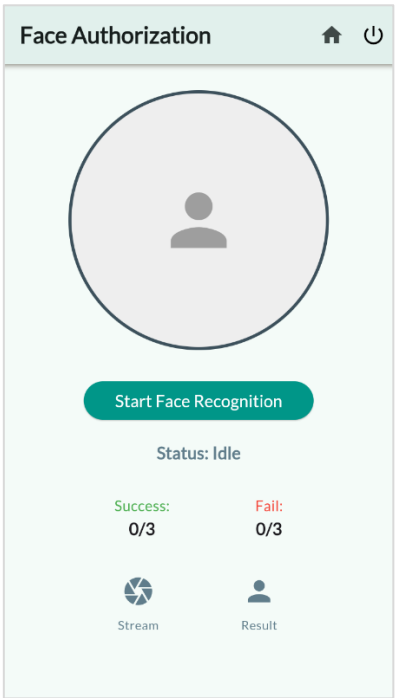


Figure 5.5.3 Face authentication page

The webcam begins capturing live video frame of the driver. These frames are processed by the Raspberry Pi, and the results are sent to mobile application. On the application interface, user will receive real-time feedback, including the authentication status and face detection result. The driver identity is verified by comparing the face embeddings of the captured image with the embeddings store in database. If the driver achieve confidence score more than 90% for three consecutive times, then the driver is granted access. A "Continue to Dashboard" button will be available. User may press it to proceed to driver monitoring page. On the other hand, if driver fail three consecutive times, access is denied, and users are required to restart the authentication process again.

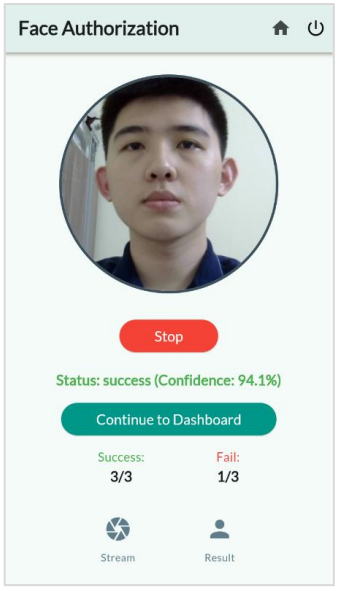


Figure 5.5.4 Authentication success

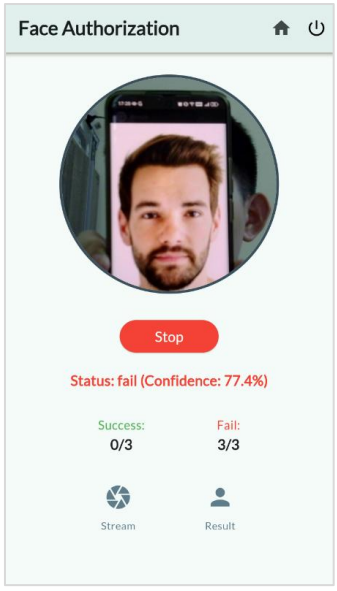


Figure 5.5.5 Authentication fail

Figure 5.5.6 LCD display when success

Figure 5.5.7 LCD display when fail

In the driver monitoring page, the main interface will provide real-time status updates on the driver condition. There are four key parameters including Eye Status, Yawn Status, Head Status

and Alcohol Status. Each of this parameter is monitored continuously and provide real time feedback. When drowsiness or alcohol detected, the states are updated on the screen with different text and colour. Raspberry Pi will trigger alert by activate the buzzer and LCD Display.

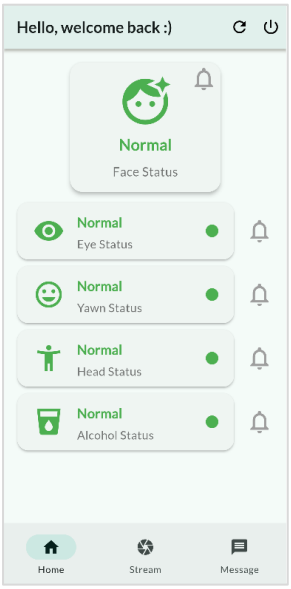


Figure 5.5.8 Monitoring page

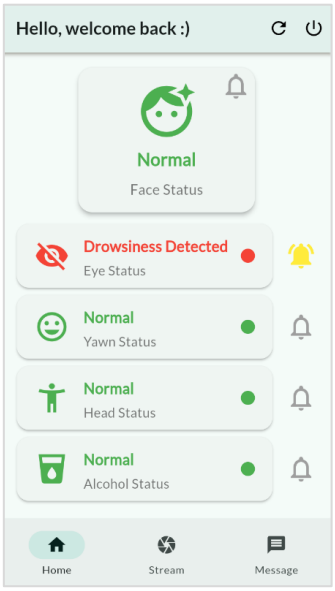


Figure 5.5.9 Drowsiness detected

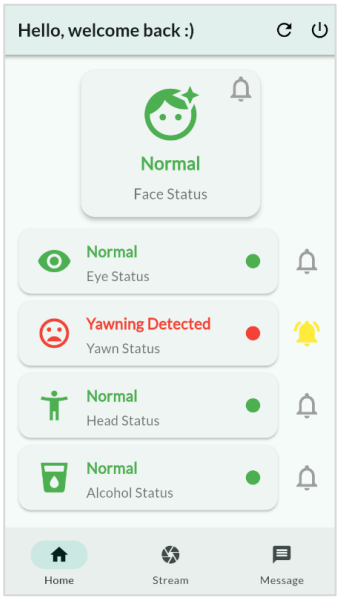


Figure 5.5.10 Yawning detected

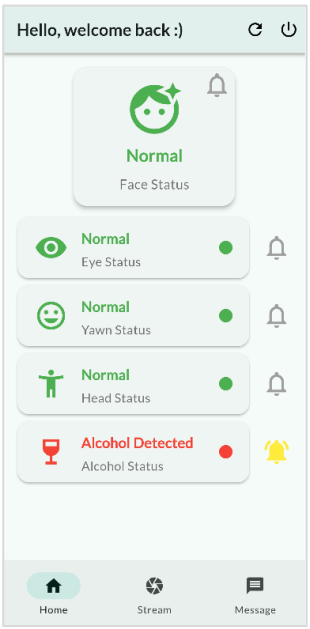


Figure 5.5.11 Alcohol detected

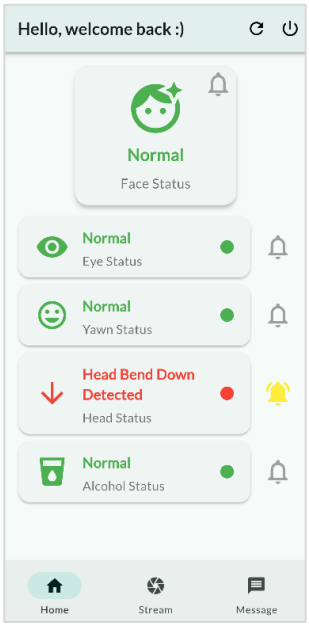


Figure 5.5.12 Head bend down detected

Figure 5.5.13 LCD display when alcohol detected

Figure 5.5.14 LCD display when drowsiness detected

The face status serves as the primary control for other detection modules. If face is detected, Raspberry Pi will continue the monitoring and update eye status, yawn status and head status. However, if no face is detected within the camera frame, the system will stops updating these parameters.

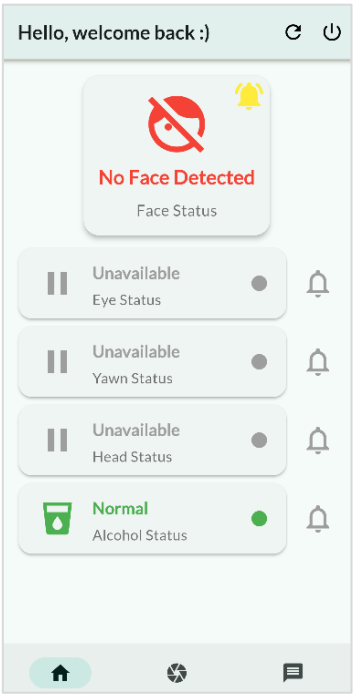


Figure 5.5.15 No face detected

The reason is because eye closure, yawning and head bending relies on face landmark and cannot be measure without a valid face region. The alcohol status is the only parameter that active all the time as it operates independently. This design is essential to ensure the monitoring system provides accurate feedback when driver face is not visible to camera. Other than that,

the mobile application also integrated with useful features. First, is the “Message” page. When any drowsy expression detected, all the results are display in the message page for later revise. These messages come with drowsy image, reason and date. For eye closed detection, an additional information such as the prediction and accuracy are included. This is to reflect the eye-closed detection model that was implemented earlier. The data displayed serve as a session storage, where all the data is disposed when Raspberry Pi shutdown.

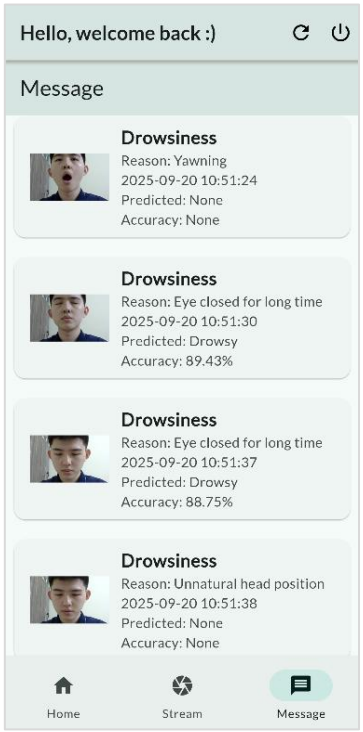


Figure 5.5.16 Message page

Besides, a “Stream” page is also developed. This page is a good measurement to address no face detected issue. In real driving conditions, there may be situations where the system may report “No Face Detected” in the mobile app. This can happen if the camera is not properly mount or driver adjust their seating position. When this happen, driver can quickly navigate to stream page and press “Open Stream in Browser” button to visually inspect what the camera sees. After that, the driver can reposition the camera to ensure his face is within the camera frame. By integrating this features, it make the system more user-friendly and practical for real world usage.

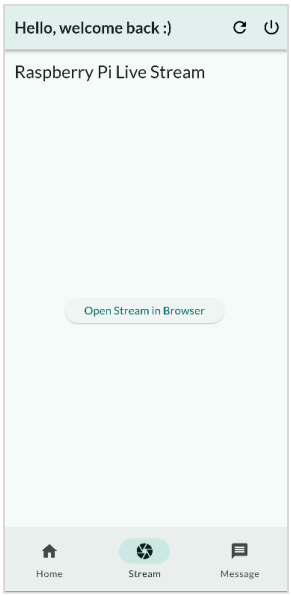


Figure 5.5.17 Stream page

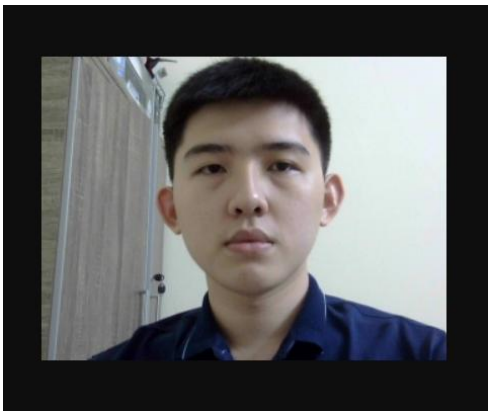


Figure 5.5.18 Streaming in browser

Last but not least, the shutdown feature. This feature aims to provide driver a convenient way to terminate the program easily when reach their destination. When press the power off button located at the upper right corner, driver is prompt to either cancel or end the session. With this feature, driver can remote control Raspberry Pi without the need of manual intervention. Also, shutdown feature prevents file corruption and hardware damage.

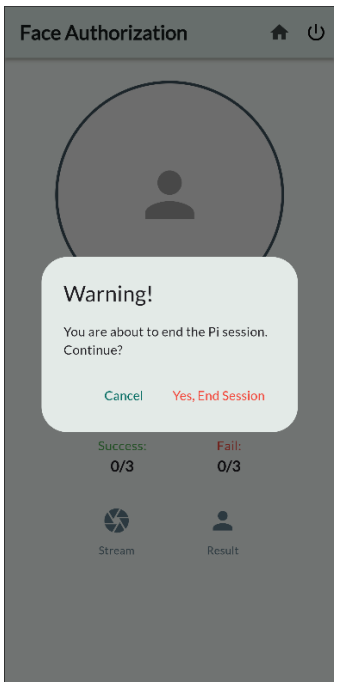


Figure 5.5.19 Shutdown feature

5.6 Implementation Issues and Challenges

During the development of drowsiness detection system, several implementation issues and challenges were encountered. First is hardware dependencies. As mentioned earlier, the LCD display relies on I2C channel to function, and it is very sensitive to the wiring. If the prototype was moved or shaken a bit, even slightly loose connections could interrupt the communication. For MQ3 alcohol gas sensor, the sensor is sensitive to environment condition such as air flow. The sensor works well in an environment with no air movement. However, the performance degrade when air flow exist, where the alcohol need to get very near to the sensor to get detected.

From software perspective, the stable communication between Raspberry Pi and mobile app was another challenge. This is because the application communicates over REST API to control or receive message from Raspberry Pi. So, any changes of IP address especially after shutdown may disrupt the connection. When this happen, all the IP address used in the mobile app will need to change to the new one manually.

Other than that, since the program is executed through VNC, the network connectivity is another limitation. The performance of the system is highly dependent on the stability and strength of Wi-Fi connection. When the network is stable, Raspberry Pi can process smoothly, and mobile application can receive timely updates. However, when the connection is weak, the VNC session become laggy. This will affect the communication between Raspberry Pi and mobile app, as REST API request could be delay. For example, Raspberry Pi detect yawning, but mobile application may not update on time.

5.7 Concluding Remark

By using Raspberry Pi as the main control unit, the system was able to process facial landmarks to detect eye closure, yawning, head posture, and alcohol correctly. The integration of mobile application along with Raspberry Pi also improved usability by providing driver authentication, monitoring, and real-time notifications. Although challenges were encountered, such as hardware wiring sensitivity and network connectivity. These challenges have highlighted the practical considerations in real-world application. Overall, the prototype successfully showed its effectiveness in detecting drowsiness and distraction, issuing timely alerts, and logging events.

Chapter 6

System Evaluation and Discussion

6.1 System Testing and Performance Metrices

The system testing evaluate how well the driver monitoring system performs under common real-world scenarios. It covers face authentication, drowsiness detection and alcohol detection. The test cases are designed to reflect practical variations including different driver, lighting conditions, face occlusion, driver expression head angles.

6.1.1 Face authentication

The test carried out for face authentication aimed to ensure the system is able to correctly identifies authorized driver and rejects unauthorized one under variation of environment.

Test Cases:

- **Different driver:** Evaluate system accuracy when an authorized or unauthorized face is captured.
- **Lighting condition:** Test the system performance under different lighting condition such as normal or low light.
- **Face occlusion:** Evaluate detection when driver face is partially covered by spectacles or face mask.
- **Facial expression:** Test recognition when driver is showing other expression like smiling or frowning.
- **Head angles:** Check the system accuracy when driver turns or tilts head.

6.1.2 Drowsiness Detection

Drowsiness detection is tested to ensure the system accurately identifies signs of driver fatigue under common driving conditions.

Test Cases:

- **Face Detection:** Evaluate other parameters when no face is detected
- **Eyes closed:** Detect when the driver closes eyes for a short period.
- **Long eye closure:** Detect prolonged eye closure.
- **Yawning:** Detect when the driver yawns naturally.
- **Head bent down:** Detect when the driver bends head forward (tired posture).
- **Wearing spectacles:** Ensure glasses do not affect detection accuracy.
- **Lighting condition:** Test performance under normal and low light.

6.1.3 Alcohol Detection

Alcohol detection is tested to ensure the system can correctly identifies the presence of alcohol and avoid false alerts. The test case evaluate how well the alcohol sensor responds to variations in distance and air flow.

Test Cases:

- **Distance:** Check whether the sensor is able to detect the presence of alcohol at varying distance.
- **Air flow:** Test the sensor performance under different air flow condition.

6.2 Testing Setup and Result

6.2.1 Testing Setup

Before the system start operating, the prototype is securely mounted to simulate in a realistic car setup. The prototype is expected to place on the dashboard near to the driver. This is to ensure sensor able to detect alcohol from the driver breath and buzzer can produce audible alert. Wires are neatly routed along the dashboard to avoid interference driver movement. For the webcam, it is mount properly in front of the driver. The camera module is mounted lower than the driver eye level so that it will not block the driver's view. The stream function in the mobile app is used to ensure the camera fully capture driver face. After the prototype is properly setup, the system is ready to run.

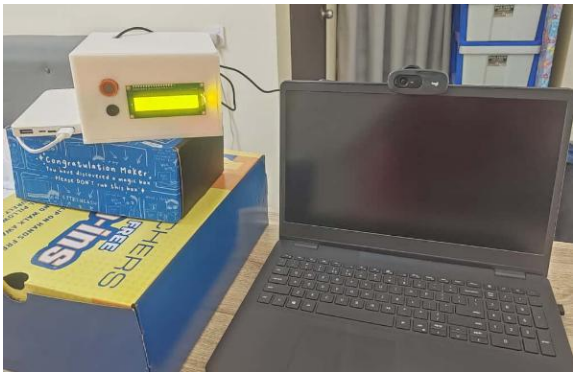
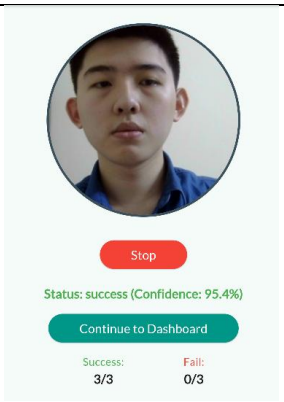
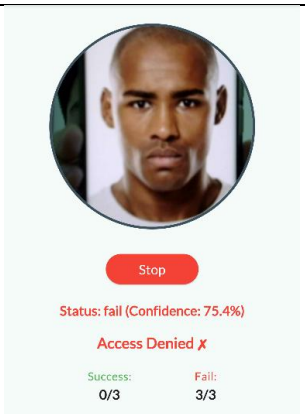
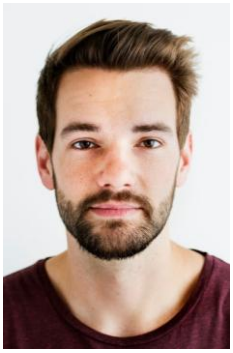
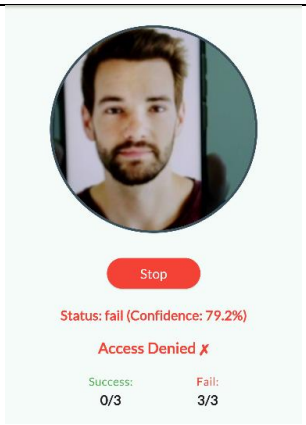
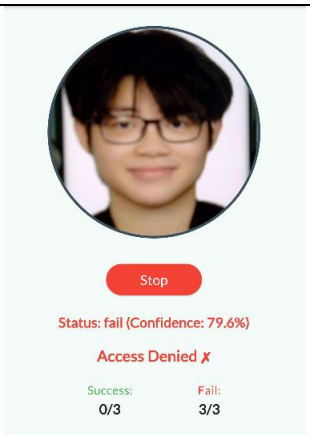
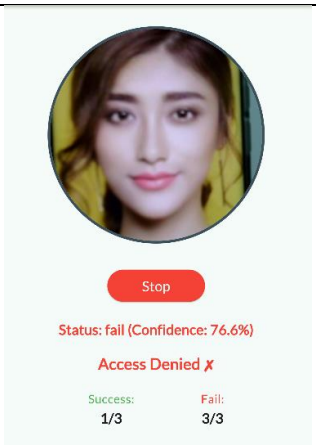
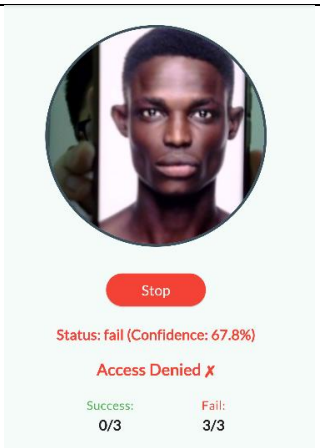


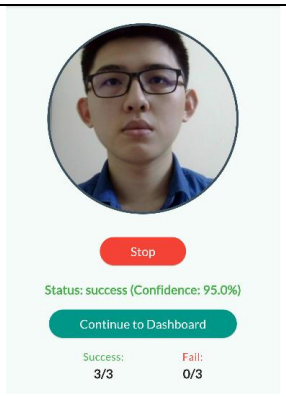
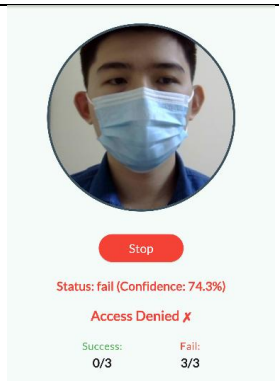
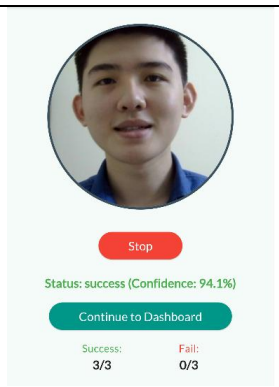
Figure 6.2.1.1 Component Setup

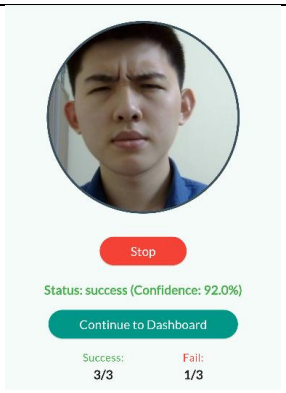
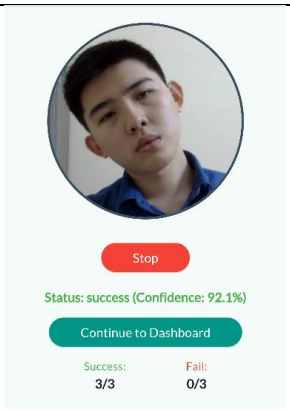
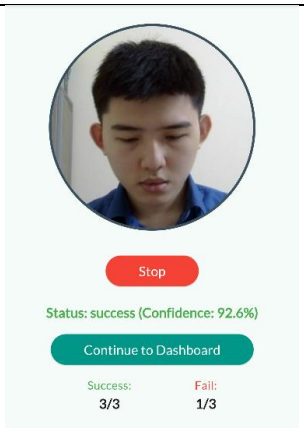
6.2.2 Face Authentication Result

Driver Image	Output	Result (> 90%)
Valid driver 		Pass
Stranger #1 		Fail

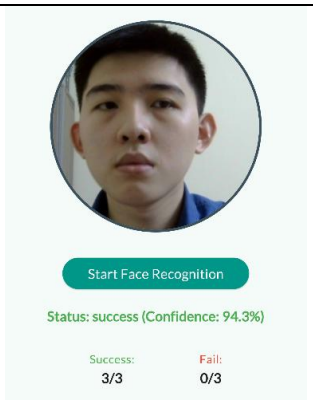
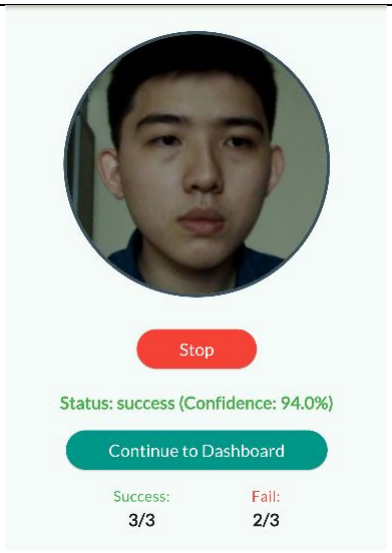
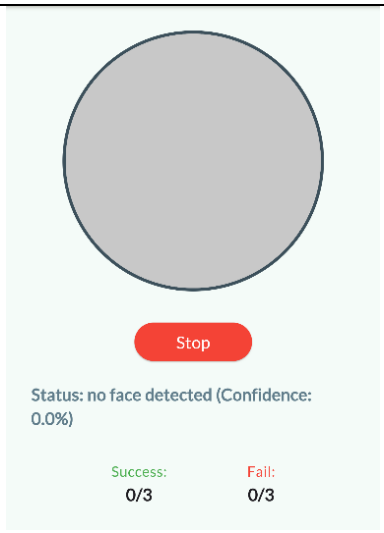
Stranger #2 		Fail
Stranger #3 		Fail
Stranger #4 		Fail
Stranger #5 		Fail

6.2.3 Face Occlusion, Different Expression and Different Head Angles Result

Driver Image	Output	Result (> 90%)
With Spectacles 		Pass
With face mask 		Fail
Smiling face 		Pass

Frowning 		Pass
Head tilt 		Pass
Look downward 		Pass

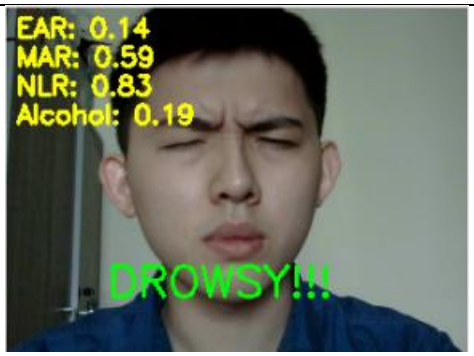
6.2.4 Different Lighting Condition Result

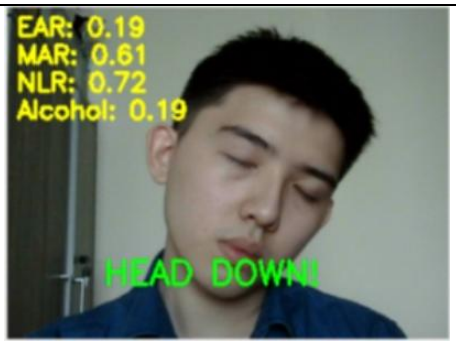
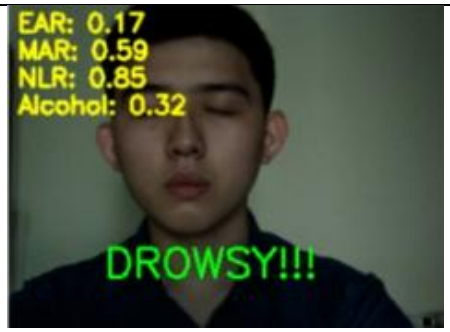
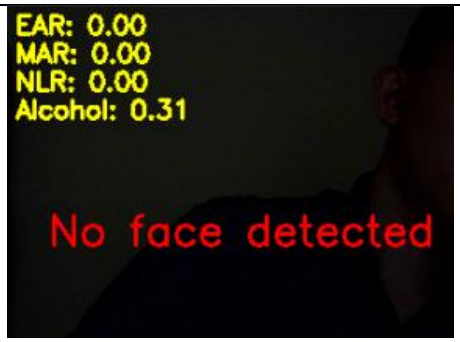
Driver Image	Output	Result (> 90%)
Normal Light 		Pass
Moderate Light 		Pass
Low Light 		Fail

As a summary, the face authentication system was evaluated under various common scenarios to determine its accuracy and robustness. The system performed well under normal lighting conditions and correctly recognized all authorized users and reject all unauthorized user. This confirm the system's reliability in preventing unauthorized access. Minors degrade in confidence value when the driver head is tilt and looking downward, but overall recognition still remained effective. For facial expressions like smiling or frowning, although the confidence slightly dropped but it not really affect recognition accuracy. Overall, the face recognition module is reliable under normal conditions but has limitations under poor lighting or when the face is partially obstructed.

6.2.5 Drowsiness Detection Result

Normal light:

Driver Image	Output	Result
Eye Closed		Pass
Yawning		Pass

Head bend down		Pass
Head tilt		Pass
Wearing spectacles		Pass
Moderate Light		Pass
Low light		Fail

The drowsiness detection system was evaluated under several scenarios including eye closure, yawning, head tilt, head bend down, and different lighting conditions. The system show reliable result across the test cases. Drowsy expression such as eye closed for fixed period, yawning and head bending were accurately identified. Wearing spectacles do not affect the detection accuracy. Under light condition test, the system show consistent performance for normal and moderate light condition. However, under low light condition, as the camera is not able to detect driver face, this drowsiness detection could not provide a valid result. Overall, the drowsiness detection system show high accuracy and successfully detecting all driver fatigue signs.

6.2.6 Alcohol Detection Result

By considering the driving cabin, the maximum distance between sensor and alcohol sensor is set to 1m only.

No airflow:

Distance between sensor and alcohol (cm)	Result
10	Pass
30 - 50	Pass
100	Pass

With airflow:

Distance between sensor and alcohol (cm)	Result
10	Pass
30 - 50	Fail
100	Fail

The alcohol detection module was evaluated under two conditions including varying distance and presence of airflow. Under no airflow, the sensor is able to detect presence of alcohol up to distance of 1m. This demonstrate good sensitivity and coverage. However, the sensor fail to detect the presence of alcohol when airflow exist. This is due to the air can disperse alcohol vapours and reduce the sensor effectiveness. Overall, the alcohol detection show good

performance, but it is sensitive to airflow, which should be considered to repositioning the sensor for real world-application.

6.3 Project Challenges

During development phase, there are several challenges that need to be featured on. First, the system relied heavily on the camera for both face recognition and drowsiness detection. In this case, the camera is sensitive to environment condition. For example, poor lighting, glare from sunlight, or low light conditions often reduced the result accuracy. Besides, hardware integration was not easy since the Raspberry Pi, camera, alcohol sensor, buzzer, and LCD display required space and a stable power supply. Raspberry Pi's limited processing power sometimes can cause delays when multiple modules ran together. Another issue was setting the right thresholds for drowsiness detection. This is because, if strict values is used, it may lead to false alarms. On the other hand, when the threshold set is too low, it may missed real fatigue signs. This required a lot of trial and error. Additionally, driver's face also had to stay within the camera view. If they bent down, leaned back, or turned away, the system may not detect it.

On the alcohol detection side, airflow may come from air conditioning or open windows and reduced sensor accuracy. This is reasonable as strong airflow will disperse the alcohol concentration in the air making the presence of alcohol undetectable. Finally, testing was mostly done in a controlled setup rather than a real car. Many assumptions was made to ensure optimize results were obtained, this will undeniably leave some uncertainty in real-world performance.

6.4 Objectives Evaluation

Objectives	Evaluation
Develop a Driver Authentication System	This objective was successfully achieved. The system is able to recognize the authorized driver before the monitoring begins, ensuring that only the registered driver can use the vehicle. It worked well in most test cases, although performance dropped in low-light or when the face was covered, which shows that improvements can be made in future versions.

Implement a Real-Time Monitoring System to Detect Driver Drowsiness	This objective was also achieved. The system can track the driver's eye closure, yawning, and head movements in real time and classify if there are any signs of drowsiness. During deployment, the system show good result and trigger alert when fatigue signs were detected. As a summary. the system is responsive and accurate which is practical in real driving situations.
Implement Alcohol Impairment Detection System	This objective was partially achieved. The system was able to detect alcohol from the driver's breath at a reasonable distance. However, the sensor performance degraded when there was airflow.

6.5 Concluding Remark

To conclude, the driver monitoring system successfully achieves all main objectives. It can authenticate the authorized driver, monitor drowsiness in real time, and detect presence of alcohol. The system works well under normal conditions, with face recognition accurately identifying drivers, capture fatigue signs, and detect alcohol through driver breath. However, some limitations were observed during testing. For example, face authentication was less accurate under low-light conditions or when the driver wore a mask, and the alcohol sensor's readings could be affected by airflow. Despite these minor issues, the system demonstrated its potential to enhance driver safety by providing timely alerts and ensuring only authorized drivers operate the vehicle.

Chapter 7

Conclusion and Recommendation

7.1 Conclusion

The driver monitoring system was designed to combine face authentication, drowsiness detection and alcohol detection into one working prototype. Through development and testing, the system showed reliable results, especially detecting driver fatigue and alcohol presence under controlled conditions. The drowsiness detection module was able to recognize signs including frequent eye closure, yawning and head positioning. Despite the system sometimes struggled with lighting, it still proved to be a reliable foundation for monitoring driver alertness. The alcohol detection module also worked effectively in still air conditions and manage to detect alcohol presence up to a distance of one meter. Besides, the face authentication also provided another layer of security. By verifying the driver's identity before allowing access, it can prevent unauthorized access to vehicle issue. This module performed well in most controlled tests, although its accuracy decreased when the face was not fully in view or when lighting conditions were poor.

As a summary, the project successfully met all objectives. The design driver monitoring system is indeed a good safety solution for driver. Of course, the system was not free from challenges. For example, camera dependency, threshold calibration, hardware limitations, and environmental interferences were the biggest challenges. With further refinement, especially through real-world testing and better hardware, the system could be developed into a more reliable driver safety tool.

7.2 Recommendation

For future work, several improvements can be made to make the system more solid. First, the drowsiness detection module should be deployed in various environments. For example, during nighttime driving or long highway trips. To solve the poor lighting condition issue, it is essential to add infrared cameras so that face and eye detection become more consistent. Furthermore, advanced deep learning models could replace threshold-based detection. Since they are better at handling small differences in driver behaviour and reduce likeliness to trigger

false alarms. Next, the alcohol detection module should be enhanced to deal with airflow issues. A possible improvement is to use a more sensitive sensor or place the sensor closer to the driver's mouth area, such as near the steering wheel. Another recommendation is to combine breath analysis with the sensor, so the system only takes input when it detects air blown by the driver.

On the hardware side, moving from Raspberry Pi to a more powerful embedded board or edge AI device could reduce processing delays when running all modules together. A more compact design should also be considered, as real vehicle cabins have limited space. In addition, the power supply system should be tested in real cars to make sure it can handle vibration, heat, and long usage. The face authentication module could also be improved by using multi-angle recognition. This would reduce the problem when the driver turns their head or shifts position. Using 3D face recognition or combining it with voice authentication could also make the system more secure and reliable.

Lastly, there is still area of improvement for the mobile application. Currently the system is just simply authenticating the driver without having any safety measurement. In future, possible measurement can be implemented is by sending notification or calling the valid driver when intruder detected. Besides, the current application does not equip with proper databases to store the detection result. This issue could be solved by using a cloud-hosted database such as Firebase or MongoDB, which allow results to be stored and accessed more effectively. Also, the mobile application can be further improved by giving it the ability to control Raspberry Pi directly. For example, the app can start the system remotely with only single clock, so the driver does not need to manually set up the hardware every time. This measurement make the system more practical and user-friendly, especially for real-world use.

REFERENCES

[1] S. Jeyaraman, “Malaysia ranked third in highest road fatalities,” *thesun.my*, Jul. 27, 2023. Available:
https://thesun.my/style-life/going-viral/malaysia-ranked-third-in-highest-road-fatalities-HL11290326

[2] “The Safest Roads in the World Report | FINN,” *FINN*.
Available:
https://www.finn.com/en-US/press/the-worlds-safest-roads

[3] K. Blake, “Everything you need to know about drowsiness,” *Healthline*, Aug. 01, 2019. Available:
https://www.healthline.com/health/drowsiness

[4] A. K. Biswal, D. Singh, B. K. Pattanayak, D. Samanta, and M.-H. Yang, “IoT-Based Smart Alert System for drowsy driver detection,” *Wireless Communications and Mobile Computing*, vol. 2021, pp. 1–13, Mar. 2021.
Available:
https://doi.org/10.1155/2021/6627217

[5] P. Kielty, Dilmaghani, Mehdi Sefidgar, C. Ryan, J. Lemley, and P. Corcoran, “Neuromorphic Sensing for Yawn Detection in Driver Drowsiness,” *arXiv (Cornell University)*, Jan. 2023.
Available:
https://doi.org/10.48550/arxiv.2305.02888

[6] R. Jabbar, M. Shinoy, M. Kharbeche, K. Al-Khalifa, M. Krichen, and K. Barkaoui, “Driver Drowsiness Detection Model Using Convolutional Neural Networks Techniques for Android Application.” Accessed: May 09, 2024. [Online].
Available:
https://arxiv.org/pdf/2002.03728

[7] “Real-Time Drowsiness Alert System from EEG Signal Based on FPGA,” *IEEE Conference Publication | IEEE Xplore*, Dec. 22, 2021.
Available:
https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=9718788&casa_token=kTzzgPEnq8gAAAAA:raNBCm-3TulKEoQaL0-IwRnfIXcg23zKxqks6v9rbKC4hx5wPOhlyFwl3NR4oV0tfAGYFAmwHy6R&tag=1

[8] M. Arunasalam, N. Yaakob, A. Amir, M. Elshaikh, and N. F. Azahar, “Real-Time Drowsiness detection system for driver monitoring,” *IOP Conference Series Materials Science and Engineering*, vol. 767, no. 1, p. 012066, Feb. 2020,
Available:
doi: 10.1088/1757-899x/767/1/012066

[9] “Non-Interference driving fatigue detection system based on intelligent steering wheel,” *IEEE Journals & Magazine | IEEE Xplore*, 2022.
Available:
https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=9918061&casa_token=3uFSpb_sMdQAAAAA:u922S2ExgMd4EC1FABvAKYpV-9p68NFAie3nqbqGGkU8QVRh_kiB5hg6L9_6zi9IUz2lvZVHe5DR

[10] Raspberry Pi, “Raspberry Pi 4 Model B specifications,” *Raspberry Pi*.
Available:
https://www.raspberrypi.com/products/raspberry-pi-4-model-b/specifications/

[11] T. Agarwal, “MQ3 Alcohol Sensor : Datasheet , Working & Its Applications,” *ElProCus - Electronic Projects for Engineering Students*, Jun. 13, 2022.
Available:
https://www.elprocus.com/mq3-alcohol-sensor/

[12] “Logitech C270 HD Webcam, 720p Video with Noise Reducing Mic,” *www.logitech.com*.
Available:
https://www.logitech.com/en-my/products/webcams/c270-hd-webcam.960-000627.html

[13] Microchip Technology Inc., “MCP3004/3008,” 2008.
Available:
https://cdn-shop.adafruit.com/datasheets/MCP3008.pdf

[14] “Handson Technology User Guide Active Buzzer Module -Low Level Trigger.” Available:
https://www.handsontec.com/dataspecs/module/active%20buzzer%20module.pdf

[15] “I2C 1602 Serial LCD for Arduino & RPI,” *Cytron Technologies Malaysia*, 2025. Available:
https://my.cytron.io/c-electronic-components/p-i2c-1602-serial-lcd-for-arduino-and-rpi?gclid=Cj0KCQjw8p7GBhCjARIsAEhghZ0oK6y7gvBrrCyJ2U0VUuzHQHArWJQ2eWF-x-oGU-BwW4jKgxRtZ6AaApf_EALw_wcB (accessed Sep. 21, 2025).

[16] M. G. Home, “20000 mAh Redmi Fast Charge Power Bank,” *Mi Global Home*, 2025. Available:
https://www.mi.com/global/product/20000mah-redmi-fast-charge-power-bank/specs/
(accessed Sep. 21, 2025).

[17] R. Tripathi, “What are Vector Embeddings?,” *Pinecone*, Jun. 30, 2023.
Available:
https://www.pinecone.io/learn/vector-embeddings/

APPENDIX

Sample MCP3008 ADC Dataset

MICROCHIP

MCP3004/3008

2.7V 4-Channel/8-Channel 10-Bit A/D Converters with SPI Serial Interface

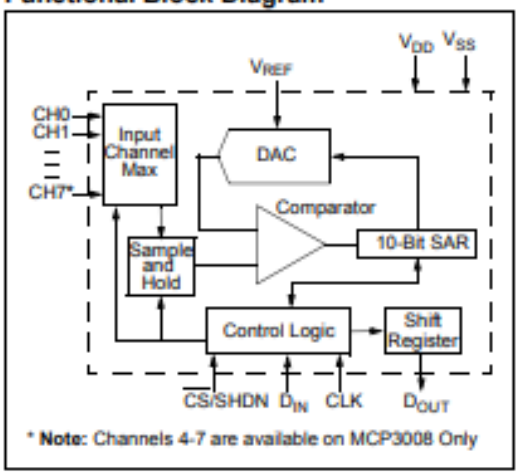
Features

- 10-bit resolution
- ± 1 LSB max DNL
- ± 1 LSB max INL
- 4 (MCP3004) or 8 (MCP3008) input channels
- Analog inputs programmable as single-ended or pseudo-differential pairs
- On-chip sample and hold
- SPI serial interface (modes 0,0 and 1,1)
- Single supply operation: 2.7V - 5.5V
- 200 ksps max. sampling rate at V_{DD} = 5V
- 75 ksps max. sampling rate at V_{DD} = 2.7V
- Low power CMOS technology
- 5 nA typical standby current, 2 µA max.
- 500 µA max. active current at 5V
- Industrial temp range: -40°C to +85°C
- Available in PDIP, SOIC and TSSOP packages

Applications

- Sensor Interface
- Process Control
- Data Acquisition
- Battery Operated Systems

Functional Block Diagram



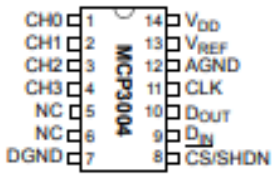
* **Note:** Channels 4-7 are available on MCP3008 Only

Description

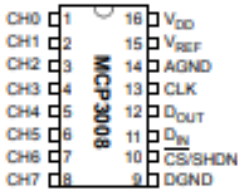
The Microchip Technology Inc. MCP3004/3008 devices are successive approximation 10-bit Analog-to-Digital (A/D) converters with on-board sample and hold circuitry. The MCP3004 is programmable to provide two pseudo-differential input pairs or four single-ended inputs. The MCP3008 is programmable to provide four pseudo-differential input pairs or eight single-ended inputs. Differential Nonlinearity (DNL) and Integral Nonlinearity (INL) are specified at ±1 LSB. Communication with the devices is accomplished using a simple serial interface compatible with the SPI protocol. The devices are capable of conversion rates of up to 200 ksps. The MCP3004/3008 devices operate over a broad voltage range (2.7V - 5.5V). Low-current design permits operation with typical standby currents of only 5 nA and typical active currents of 320 µA. The MCP3004 is offered in 14-pin PDIP, 150 mil SOIC and TSSOP packages, while the MCP3008 is offered in 16-pin PDIP and SOIC packages.

Package Types

PDIP, SOIC, TSSOP



PDIP, SOIC



POSTER

Raspberry Pi based Cyber Physical System for Driver Monitoring

UTAR
UNIVERSITI TUNKU ABDUL RAHMAN

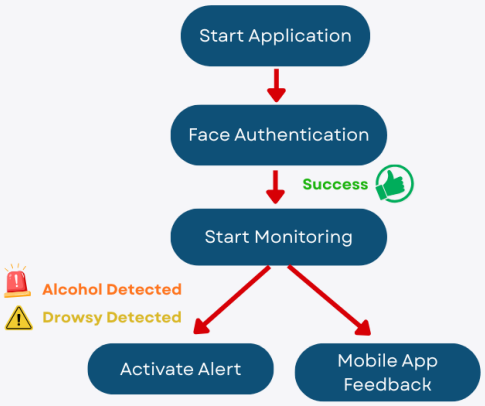
Introduction

Nowadays, the number of accidents have dramatically increase due to drowsiness problem. Therefore, an invention of developing a driver monitoring system is essential to keep driver for safe driving. Here, a prototype capable to detect driver drowsiness status is develop with advance technique. Not only that, but the prototype is equipped with other interesting functionality such as driver authentication, drunk detection and vehicle tracking features that will surely amazed you.

Our Goal

- Implement a real time drowsiness detection
- Implement alcohol detection system
- Develop a driver monitoring application
- Develop a driver authentication system

How it works?



Strengths of proposed method

In compare with existing system, our system...

- **Support real time** - drowsiness & drunkness detection
- **Prevent car theft** - face authentication
- **Easy to use** - User friendly monitoring application
- **Better driver experience** - use camera, no external wearable need to mount on driver.

Conclusion

Checklist:

1. Facial Recognition
2. Driver Drowsiness Detection
3. Alcohol Detection
4. Monitoring Application Development

Project Developer: Chee Hao Jie
Project Supervisor: Dr Teoh Shen Khang

A pause from the road might save your life